

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное автономное образовательное учреждение
высшего образования «Самарский национальный исследовательский
университет имени академика С.П. Королева»
(Самарский университет)

Институт _____ информатики и кибернетики _____

Кафедра _____ программных систем _____

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

**«РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ ДЛЯ УПРАВЛЕНИЯ
МНОГОКВАРТИРНЫМ ДОМОМ»**

по направлению подготовки 02.03.02

Фундаментальная информатика и информационные технологии
(уровень бакалавриата)

направленность (профиль) «Информационные технологии»

Обучающийся _____ И. И. Иванов
(подпись, дата)

Руководитель ВКР

доцент каф. ПС, к.т.н., доцент _____ Д. А. Попова-Коварцева
(подпись, дата)

Нормоконтролёр _____ Е.В. Сопченко
(подпись, дата)

Самара 2024

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное автономное
образовательное учреждение высшего образования
«Самарский национальный исследовательский университет
имени академика С.П. Королева»

Кафедра _____ программных систем _____

УТВЕРЖДАЮ
Заведующий кафедрой

_____ С.В. Востокин
« ____ » _____ 2024 г.

задание на выпускную квалификационную работу (ВКР)

обучающемуся _____ Иванову Ивану Ивановичу
группы _____ 6413-020302D _____

1. Тема работы: _____ Разработка веб-приложения для управления
многоквартирным домом _____
утверждена по университету от «25» апреля 2024 г. № 223-Т _____

2. Перечень вопросов, подлежащих разработке в ВКР:

- 1) Провести анализ предметной области: рассмотреть технологии разработки веб-приложения многоквартирным домом _____
- 2) Сделать обзор систем-аналогов в области (рассмотреть существующие программные средства, предназначенные для управления многоквартирным домом) _____
- 3) Разработать проект системы с использованием методологии UML _____
- 4) Разработать и реализовать информационное и программное обеспечение _____
- 5) Провести тестирование и отладку веб-приложения для управления многоквартирным домом _____

4. Дата выдачи задания: «25» апреля 2024г.

5. Срок представления на кафедру законченной ВКР: «03 »июня 2024г.

Руководитель ВКР

к.т.н., доцент,

доцент кафедры программных систем _____ Д.А. Попова-Коварцева
« 25 » 04 2024 г.

Задание принял к исполнению

_____ И.И. Иванов
« 25 » 04 2024 г.

РЕФЕРАТ

Пояснительная записка 90с, 48 рисунков, 19 таблиц, 31 источник, 2 приложения.

Графическая часть: 25 слайдов презентации PowerPoint.

УПРАВЛЕНИЕ МНОГОКВАРТИРНЫМ ДОМОМ, СОБСТВЕННИК, УПРАВЛЯЮЩАЯ КОМПАНИЯ, КВИТАНЦИЯ, МЕССЕНДЖЕР, ЛИЧНЫЙ КАБИНЕТ, КЛИЕНТ-СЕРВЕРНАЯ АРХИТЕКТУРА.

Объектом автоматизации является управление многоквартирным домом.

Цель работы – разработать веб-приложение для управления многоквартирным домом для передачи данных между управляющей компанией и собственниками, для обеспечения более быстрого и структурированного обмена данными между управляющей компанией и собственниками.

В процессе работы было разработано веб-приложение, позволяющее управляющей компании оперативно получать и передавать информацию собственникам многоквартирного дома. Система предоставляет возможность отправки и получения квитанций о коммунальных платежах, создание и просмотр объявления, создание диалога между собственниками и управляющей компании, отправку медиафайлов, текстовых и голосовых сообщений, а также поддерживать актуальной информацию о пользователе.

Система разработана на языке JavaScript с использованием библиотеки для разработки пользовательских интерфейсов React в кроссплатформенном редакторе кода Visual Studio Code и функционирует под управлением операционной системы Windows 7 и выше. Доступ к данным осуществляется с помощью СУБД PostgreSQL.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	6
1.1 Основные понятия и определения	8
1.1.1 Права и обязанности управляющей компании по содержанию многоквартирного дома	9
1.1.2 Права и обязанности собственника квартиры жилого помещения в многоквартирном доме	10
1.1.3 Виды каналов связи между УК и собственниками жилых помещений	11
1.2 Обзор существующих систем аналогов	12
1.2.1 Система «Локоло»	12
1.2.2 Система «Все соседи»	14
1.2.3 Система «Госуслуги Дом»	15
1.3 Постановка задачи	18
2 Проектирование системы	21
2.1 Выбор и обоснование архитектуры системы	21
2.2 Структурная схема системы	24
2.3 Разработка прототипа интерфейса пользователя системы	27
2.4 Разработка информационно-логического проекта системы	33
2.4.1 Язык UML	33
2.4.2 Диаграмма вариантов использования	33
2.4.3 Сценарий	35
2.4.4 Диаграмма состояний	36
2.4.5 Диаграмма деятельности	37
2.4.6 Диаграмма классов	42
2.4.7 Логическая модель данных	44
2.5 Описание комплекса программных средств	50
2.5.1 Описание языка программирования	50
2.5.2 Описание библиотеки React	51
2.5.3 Описание среды программирования	51

2.5.4	Описание СУБД.....	52
2.5.5	Описание операционной системы	52
3	Реализация системы	54
3.1	Разработка и описание интерфейса пользователя	54
3.2	Диаграммы реализации	60
3.2.1	Диаграмма компонентов	61
3.2.2	Диаграмма развертывания	63
3.2.3	Диаграмма классов	63
3.3	Физическая модель данных	64
3.4	Выбор и обоснование комплекса технических средств	68
3.4.1	Расчет объема занимаемой памяти.....	68
3.4.2	Минимальные требования, предъявляемые к системе.....	69
	ЗАКЛЮЧЕНИЕ	71
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	72
	ПРИЛОЖЕНИЕ А Руководство пользователя	75
A.1	Назначение системы.....	75
A.2	Условия работы системы	75
A.3	Работа с системой.....	75
A.3.1	Начало работы с системой	75
A.3.2	Работа с системой в режиме пользователя «Собственник»	76
A.3.3	Работа с системой в режиме пользователя «Управляющая компания».....	78
A.3.4	Основные ошибки.....	79
	ПРИЛОЖЕНИЕ Б Листинг модулей программы	82

ВВЕДЕНИЕ

Современные информационные технологии неотъемлемо вписываются в стратегии предприятий и организаций, стремящихся оптимизировать свою деятельность и улучшить управления бизнес-процессами. Все больше рутинных задач можно выполнить, открыв смартфон или ноутбук. Заказать доставку продуктов из магазина, вызвать такси, купить билет в кино или театр, забронировать отель – все эти действия давно автоматизированы и выполняются в несколько кликов. Таким образом, для людей жизнь «в режиме онлайн» постепенно стала повседневностью. Если раньше многие узнавали о предстоящих собраниях ТСЖ от знакомых или видели объявления с анонсами на улице, то теперь количество личных контактов сократилось, а на улицу граждане стали выходить реже.

Управление жилой недвижимостью – одна из самых эмоциональных и сложных сфер бизнеса. Иметь дело со множеством собственников непросто, а угодить абсолютно всем – невозможно.

Чтобы дать качественный сервис, управляющей компании нужно не только выполнять технические функции, но и поддерживать живой контакт с собственниками, опираться на их точку зрения, при этом уметь задавать вектор общего мнения. В крупных городах, где жилые комплексы насчитывают от нескольких сотен до нескольких тысяч квартир, без цифровых технологий коммуникации задачи управляющей компании решить просто невозможно.

Актуальность данной проблемы обусловлена постоянным развитием технологий и возрастающей ролью автоматизации информационных систем во всех отраслях, в том числе и в области управления жилыми зданиями. Возможности данной системы в режиме реального времени передавать необходимую информацию в обе стороны способствует повышению эффективного использования ресурсов и качественно улучшает жизнь граждан.

Во время выполнения выпускной квалификационной работы необходимо разработать веб-приложение для управления многоквартирным домом, с её помощью УК сможет в режиме реального времени передавать

необходимую информацию собственникам создавать чаты с 3-мя и более участниками, контролировать оплату квитанций на оплату коммунальных услуг, создавать объявления. «Собственник» может создавать диалоги только с 2-мя участниками, получать квитанции на оплату коммунальных услуг, общаться с соседями, просматривать объявления.

Разработка веб-приложения для управления представляет собой актуальную задачу, направленную на повышение эффективности процессов коммуникации между соседями и УК, а также на улучшение управления многоквартирным домом.

1 Описание и анализ предметной области

1.1 Основные понятия и определения

Управляющая компания (УК) – это юридическое лицо, которое занимается управлением многоквартирными домами, содержанием общедомового имущества, поддержанием надлежащего технического и санитарного состояния, поставками коммунальных ресурсов собственникам квартир [1].

Собственник – это физическое или юридическое лицо, которое осуществляет право владения, пользования и распоряжения недвижимостью в правовых рамках [2].

Компанию, которая занимается управлением МКД (многоквартирным домом) выбирают владельцы недвижимости – они вправе расторгнуть договор с одной УК и заключить его с другой. В отличие от ТСЖ, управляющая компания является сторонней организацией. Товарищество собственников жилья формируется из собственников квартир – по крайней мере, все жильцы имеют большее влияние на ТСЖ по сравнению с УК. Все УК разделяются на три типа [3]:

- управляющие – форма компании, которая самостоятельно занимается только содержанием общедомового имущества и управлением. Для оказания коммунальных услуг, технического и санитарного содержания такие компании привлекают подрядчиков, которые работают по договорам;

- гибридные – это крупные организации, которые выполняют все предписанные законодательством услуги: содержат общедомовое имущество, управляют домом, отвечают за техническое и санитарное состояние, оказывают коммунальные и другие услуги собственникам квартир;

- эксплуатационные – это менее распространённая форма компании. В этом случае УК заключает договор с ТСЖ в лице правления, председателя или напрямую с собственниками квартир. Компания занимается только

эксплуатацией, техническим и санитарным состоянием общедомовой недвижимости.

Полномочия и функции управляющей компании, её права и обязанности прописаны во множестве нормативных актов. Деятельность строго регламентирована различными документами. Основные из них:

- жилищный кодекс Российской Федерации;
- постановление Госстроя №170 от 2003 года;
- постановление Правительства РФ №354.

1.1.1 Права и обязанности управляющей компании по содержанию многоквартирного дома

Все права и обязанности управляющей компании прописаны в законодательстве.

Условно обязанности УК можно разделить на две большие группы: работы по эксплуатации многоквартирного дома (МКД) и осуществлению ремонтных работ и услуги управленческого, организационного характера.

В работы по эксплуатацию МКД, осуществлению ремонтных работ входят:

- благоустройство и озеленение прилегающей к дому территории;
- устранение аварий и их последствий в МКД;
- проведение подготовительных мероприятий перед началом зимнего периода;
- обеспечение общей и пожарной безопасности;
- текущий ремонт общедомового имущества;
- транспортировка мусора, жидких бытовых отходов;
- поддержание чистоты в подъездах, их уборка.

Услуги управленческого и организационного характера включают в себе [4]:

- учёт всех жильцов дома;
- выявление злостных неплательщиков и тех, кто потребляет ресурсы в обход счётчиков при помощи незаконного подключения;

- учёт оплаты – кто и в каком объём платит, кто нет;
- хранение, обеспечение целостности всей документации на МКД, в том числе паспортов;
- информирование собственников о тарифных планах;
- взаимодействие с предприятиями, которые обеспечивают поставку коммунальных услуг, контроль качества оказываемых услуг;
- проведение собраний жильцов, предоставление отчета о проделанной работе и плана на год не реже одного раза в год.

Управляющая компания имеет право:

- требовать от собственников квартир и арендаторов оплачивать услуги в полном объеме;
- проводить работу с должниками и истребовать уплаты долга за коммунальные услуги всеми разрешенными законом способами; направлять накопленные на счетах денежные средства на текущий ремонт, любые другие работы по обслуживанию МКД;
- приостанавливать или ограничивать поставку ресурсов за долги ЖКХ;
- использовать придомовую территорию в рамках законодательства РФ;
- использовать общедомовые помещения в МКД по прямому назначению или по своему усмотрению, если это соответствует требованиям законодательства.

1.1.2 Права и обязанности собственника квартиры жилого помещения в многоквартирном доме

Собственник жилого помещения осуществляет права владения, пользования и распоряжения принадлежащим ему на праве собственности жилым помещением в соответствии с его назначением и пределами его использования, которые установлены ЖК. К таким правам относятся:

- сдача помещений в аренду для проживания;

- безвозмездное и возмездное отчуждение имущества (обмен, дарение, наследство, продажа);
- регистрация собственника и членов его семьи в квартире, а также посторонних лиц по желанию владельца (прописка и выписка);
- использование жилплощади для трудовой деятельности (фотостудия, репетиторство, ателье);
- осуществление перепланировки в помещении.
- собственник жилого помещения обязан:
 - поддерживать помещение в надлежащем состоянии, не допуская бесхозяйственного обращения с ним;
 - соблюдать права и законные интересы соседей;
 - исправно уплачивать налог на имущество;
 - вносить денежные средства в счёт будущего капитального ремонта МКД.

1.1.3 Виды каналов связи между УК и собственниками жилых помещений

Политика государства направлена на централизацию обращений в сфере ЖКХ, повышение прозрачности работы управляющих компаний, ресурсоснабжающих организаций и государственной жилищной инспекции (п. 2 ст. 1, ст. ст. 3 и 4 Федерального закона от 21.07.2014 № 209-ФЗ). Основные виды каналов связи управляющей компании с жителями можно разделить на две большие группы: оффлайн- и онлайн- каналы оповещения.

Для оффлайн-каналов не нужен интернет. Обновление информации в них связано с материальными и временными затратами. К оффлайн-каналам относятся [5]:

- информационный стенд;
- информационные/заказные письма;
- поквартирный обход;
- листовки;

- обзвон;
- автоответчик.

Онлайн-каналы оповещения – это такие каналы оповещения, для работы с которыми необходимо подключение к сети интернет. Они позволяют оперативно доставить информацию до жителя и включают в себя такие средства [5]:

- email-рассылка;
- чат МКД в мессенджерах: Telegram, Viber;
- чат-боты;
- сервисы для УК и собственников жилья;
- группы и профили в социальных сетях;
- сайты.

К преимуществам оффлайн-каналов связи можно отнести:

- доступность собственников всех возрастной категории;
- хорошее качество восприятия информации.

К недостаткам относятся:

- низкая скорость донесения информации;
- затраты на подготовку раздаточных материалов;
- коммуникация в одну сторону.

К преимуществам онлайн-каналов связи можно отнести:

- высокая быстрого донесения информации;
- наличие быстрой обратной связь;
- мобильность;
- высокая информационная насыщенность.

К недостаткам относятся:

- требуется постоянное внимание;
- избыточный негативизм.

1.2 Обзор существующих систем аналогов

1.2.1 Система «Локоло»

«Локоло» – это бесплатный государственный сервис для жителей Республики Татарстан, а рамках которого жители могут беспрепятственно и легитимно голосовать на Общем собрании собственников, решать жилищные вопросы и общаться с соседями [6].

В «Локоло» можно создавать локальные сообщества по интересам в рамках одного дома, формировать инициативные предложения от горожен и передавать в ответственные органы, создавать голосования и опросы для улучшения взаимодействия человека и государства.

На рисунке 1 показан интерфейс системы «ЛОКОЛО».

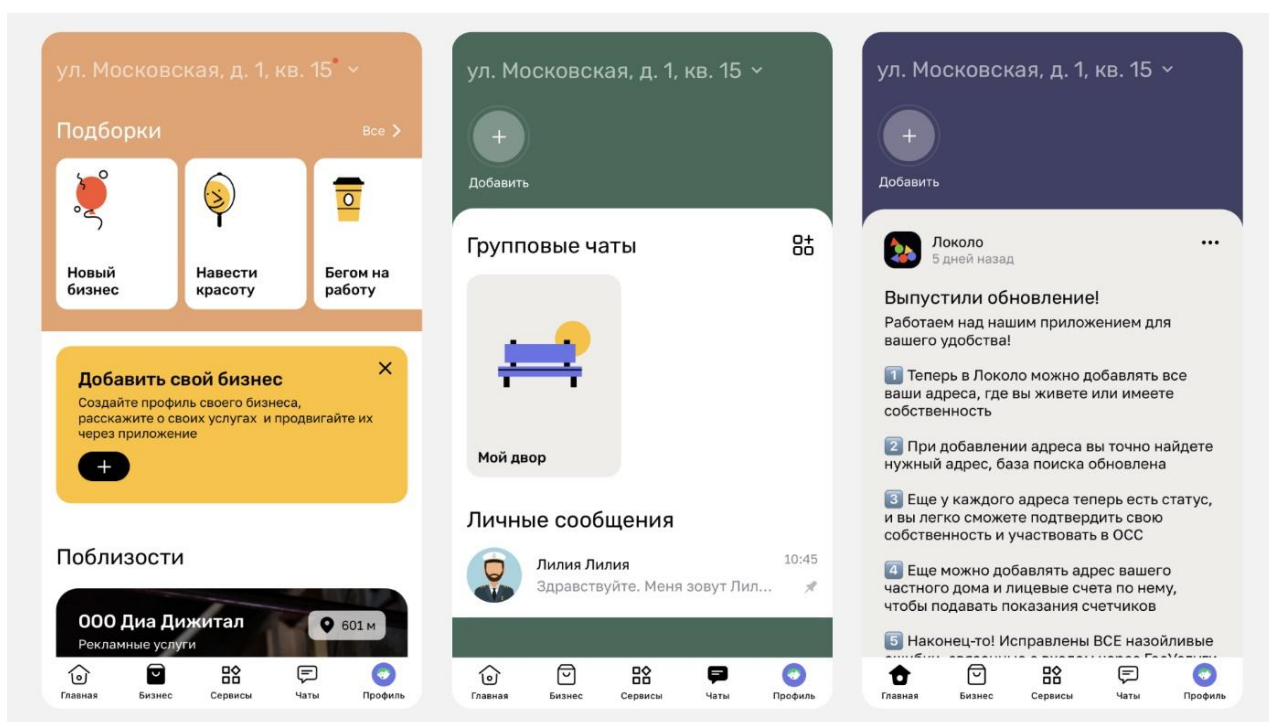


Рисунок 1 – Пример работы системы «Локоло»

К достоинствам системы относятся:

- мессенджер обеспечивает общение с другими собственниками в МКД по жилищным вопросам и в чатах по интересам;
- сервисы ЖКХ позволяют получать квитанции на коммунальные услуги прямо через приложение;
- объявления позволяют публиковать предложения своих услуг/товаров для продвижения и поиска клиентов по близости;
- личный кабинет позволяет редактировать свои данные.

К недостаткам системы относятся:

- территориальная ограниченность действия республикой Татарстан;
- частые обновления мобильного приложения;
- реклама.

1.2.2 Система «Все соседи»

«Все соседи» представляет из себя симбиоз гиперлокальной (районной) социальной сети для соседей и онлайн-биржи услуг [7].

Сервис позволяет жителям многоквартирных домов получать или оказывать ряд услуг своим соседям. Он помогает быстро и непринуждённо выстраивать коммуникации с соседями по подъезду, дому, жилому комплексу, району или даче, а также справляться с повседневными делами с помощью соседей или зарабатывать, оказывая услуги.

На рисунке 2 показан интерфейс системы «Все соседи».

Рисунок 2 – Пример работы системы «Все соседи»

К достоинствам системы относятся:

- онлайн-биржа позволяет соседям находить исполнителя, подработку, или обмениваться с соседями услугами;

- мессенджер обеспечивает общение с другими собственниками один на один;

- личный кабинет позволяет редактировать свои данные.

К недостаткам системы относятся:

- невозможность общения в чате мессенджера одновременно с 3-мя и более соседями;

- отсутствие канала связи с УК;

- отсутствие сервисов ЖКХ;

- реклама.

1.2.3 Система «Госуслуги Дом»

«Госуслуги Дом» – приложение для собственников жилья в многоквартирных домах.

«Госуслуги Дом» позволяет воспользоваться основными сервисами и электронными услугами, связанными с городским хозяйством: передать показания приборов учета водоснабжения и электроэнергии, узнать о плановых и аварийных отключениях воды, электроэнергии, газа и лифтов, пожаловаться на неполадки в доме и во дворе, провести голосование среди собственников, а также общаться с соседями [8].

На рисунке 3 показан интерфейс системы «Госуслуги Дом».

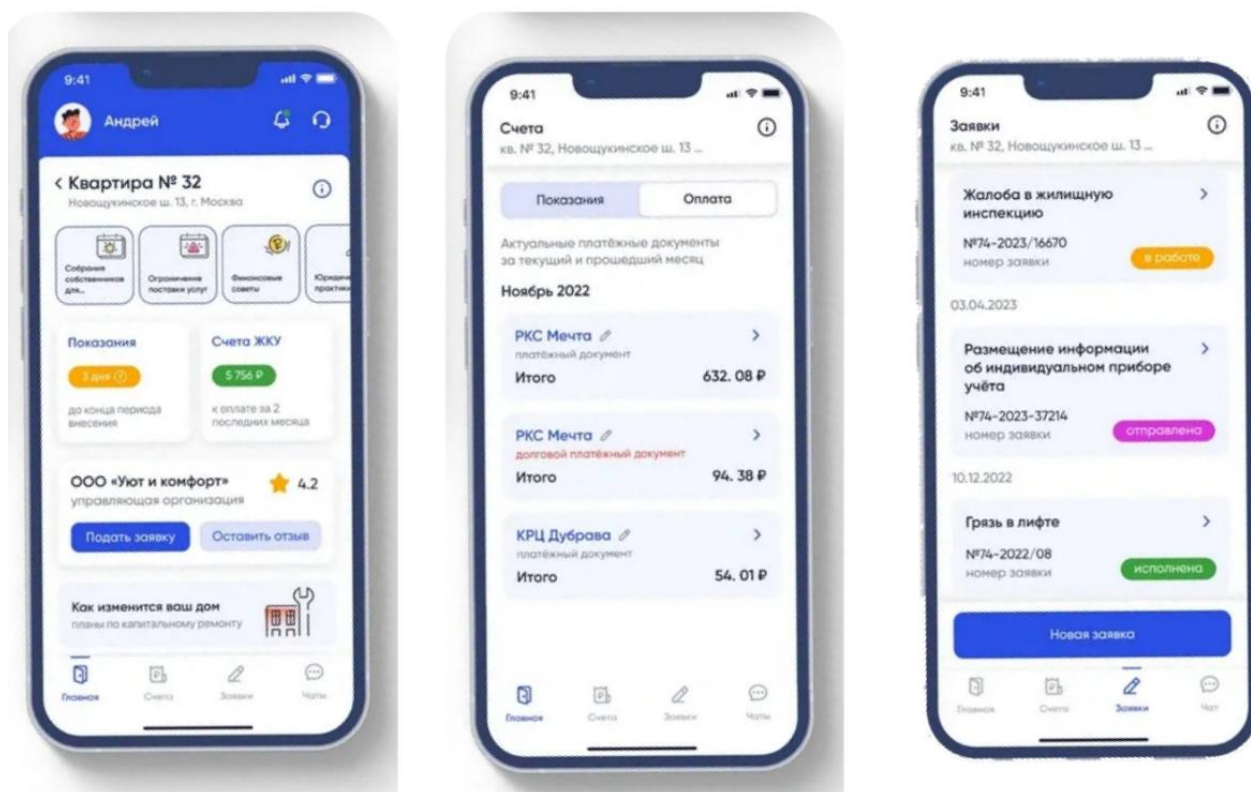


Рисунок 3 – Пример работы системы «Госуслуги Дом»

К достоинствам системы относятся:

- мессенджер обеспечивает общение с другими собственниками в МКД по жилищным вопросам и в чатах по интересам;
- сервисы ЖКХ позволяют получать квитанции на коммунальные услуги прямо через приложение;
- новости позволяют предупредить соседей об опасности в доме;
- механизм отправки показания счетчиков позволяет собственникам доставить нужную информацию УК, не выходя из дома;

К недостаткам системы относятся:

- частые обновления мобильного приложения;
- реклама.

В таблице 1 приведены характеристики систем-аналогов.

Таблица 1 – Сравнительные характеристики систем-аналогов

Название Название системы	Локоло	Все соседи	Госуслуги Дом
---	--------	------------	---------------

показателя			
1	2	3	4
Услуги ЖКХ	+	—	+
Создания чата	+	—	+
Создание диалога	+	+	+
Частые обновления мобильного приложения	+	+	+
Канал связи с УК	+	—	+
Личный кабинет пользователя	+	+	+
Реклама	+	+	+
Территориальная ограниченность субъектом	+	—	—
Понятность интерфейса (1-5)	4	3	4

1.3 Постановка задачи

Во время выполнения выпускной квалификационной работы необходимо разработать веб-приложения для управления многоквартирным домом. С его помощью УК сможет в режиме реального времени передавать необходимую информацию собственникам создавать чаты с 3-мя и более участниками, контролировать оплату квитанций на оплату коммунальных услуг, создавать объявления. «Собственник» может создавать диалоги только с 2-мя участниками, получать квитанции на оплату коммунальных услуг, общаться с соседями, просматривать объявления.

Для того чтобы начать работу с системой пользователю необходимо пройти регистрацию (ввести имя пользователя (от 2 до 30 символов), отчество пользователя (от 4 до 40 символов), фамилию пользователя (от 4 до 40 символов), логин (от 4 до 30 символов) и пароль (от 4 до 30 символов)) и авторизацию (ввести логин (от 4 до 30 символов) и пароль (от 4 до 30 символов)). Система должна проверить правильность введенных данных и в случае возникновения ошибок вывести предупредительное сообщение.

При работе с веб-приложением собственник сможет создать диалог, отправить или удалить текстовое и аудио сообщение, медиафайлы, отредактировать данные в личном кабинете, получить квитанцию на оплату коммунальных услуг, просмотреть объявления. При создании диалога пользователь должен выбрать получателя, а также написать текстовое сообщение.

При работе с веб-приложением УК сможет создать чат многоквартирного дома, отправить собственникам квитанции на оплату коммунальных услуг, рассылать объявления и общаться с собственниками в мессенджере. При создании чата УК должна выбрать собеседников, а также написать текстовое сообщение и название чата.

Вся информация, необходимая для работы системы, будет храниться в БД, ее структура разрабатывается на основе следующих сведений:

- о пользователе (имя пользователя, фамилия пользователя, отчество пользователя, логин, пароль, номер квартиры, номер телефона, сожители, последнее посещение, аватар, роль);
- о диалоге (автор диалога, партнер диалога, последнее сообщение);
- о сообщениях (автор сообщения, текст, диалог, статус получения, прикрепленные файлы, дата отправки);
- о прикрепленных файлах (автор медиафайла, название, размер, ссылка, сообщение);
- о квитанции (название, размер, ссылка, номер квартиры, статус оплаты, месяц);
- об объявлении (автор объявления, текст, прикрепленные файлы, дата отправки);
- о роли (название роли).

Необходимо обеспечить целостность БД.

Пользователь сможет получить квитанцию на оплату услуг ЖКХ, доступ к чату многоквартирного дома, отправить сообщение, редактировать свои личные данные, просмотреть сведения о разработчиках и о системе.

Таким образом, система должна решать следующие задачи:

1) функции системы:

- аутентификация пользователя в системе;
- регистрация пользователя в системе;
- отображение списка квитанций;
- отображение списка диалогов;
- отображение списка объявлений;
- отображение данных о пользователе;
- отображение справочной информации;

2) функции пользователя:

- регистрация пользователя в системе:
 - а) ввод имени пользователя;

- б) ввод отчества пользователя;
- в) ввод фамилии пользователя;
- г) ввод логина;
- д) ввод пароля;
- авторизация пользователя в системе:
 - а) ввод логина;
 - б) ввод пароля;
- создание нового диалога между двумя пользователями;
- отправка текстового сообщения;
- отправка аудио сообщения;
- отправка медиа сообщения;
- удаление сообщения;
- редактирование данных в личном кабинете;
- просмотр квитанций на оплату коммунальных услуг;
- просмотр объявлений;
- просмотр справочной информации.

3) функции управляющей компании:

- авторизация управляющей компании:
 - а) ввод логина;
 - б) ввод пароля;
- создание чата с несколькими пользователями;
- загрузка квитанций на оплату коммунальных платежей;
- отправка текстового сообщения;
- отправка аудио сообщения;
- отправка медиа сообщения;
- редактирование данных в личном кабинете;
- удаление сообщения;
- создание объявления;
- просмотр справочной информации.

2 Проектирование системы

Проектирование информационных систем всегда начинается с определения цели проекта [9]. Основная задача любого успешного проекта заключается в том, чтобы на момент запуска системы и в течение всего времени ее эксплуатации можно было обеспечить:

- требуемую функциональность системы и степень адаптации к изменяющимся условиям ее функционирования;
- требуемую пропускную способность системы;
- требуемое время реакции системы на запрос;
- безотказную работу системы в требуемом режиме, иными словами – готовность и доступность системы для обработки запросов пользователей;
- простоту эксплуатации и поддержки системы;
- необходимую безопасность.

Производительность является главным фактором, определяющим эффективность системы. Хорошее проектное решение служит основой высокопроизводительной системы.

Проектирование информационных систем охватывает три основные области:

- проектирование объектов данных, которые будут реализованы в базе данных;
- проектирование программ, экранных форм, отчетов, которые будут обеспечивать выполнение запросов к данным;
- учет конкретной среды или технологии, а именно: топологии сети, конфигурации аппаратных средств, используемой архитектуры (файл-сервер или клиент-сервер), параллельной обработки, распределенной обработки данных.

2.1 Выбор и обоснование архитектуры системы

Архитектура – это базовая организация системы, которая описывает связи между компонентами этой системы (и внешней средой), а также определяет принципы её проектирования и развития [11]. Она описывает, как

различные части приложения взаимодействуют между собой, как данные передаются и обрабатываются, а также определяет принципы, которыми руководствуется разработка приложения. Обратимся более подробно к видам архитектуры: клиент-серверная, веб-приложение, настольное приложение.

Существуют различные виды архитектур, далее рассмотрены несколько основных:

- локальная – при такой архитектуре не требуется обмен данными, все вычисления и необходимые данные находятся на устройстве пользователя;

- файл-серверная – при такой архитектуре устройство пользователя соединено с файл-сервером по сети. Файл-сервер – компьютер, хранящий данные в файлах;

- клиент-серверная – при такой архитектуре сервер не только хранит файлы, но и реализует часть функционала программной системы, например, выполняет процедуры СУБД.

Клиент-серверная архитектура (КСА) – это модель организации вычислительных систем, в которой задачи распределены между клиентами и серверами [11]. В такой архитектуре клиент, обычно являющийся пользователем или программой, запрашивает услуги или ресурсы у сервера, который отвечает на запросы, предоставляя необходимые данные или функциональность.

Клиент-серверные архитектуры могут быть разделены на различные виды в зависимости от числа «звеньев» или уровней, на которые они разбиты. Два из наиболее распространенных видов клиент-серверных архитектур – это двухзвенная (2-Tier) и трехзвенная (3-Tier) архитектуры.

Двухзвенная архитектура – это классический тип клиент-серверной архитектуры, включающий два основных компонента: клиента и сервер [11]. Клиентский компонент обеспечивает пользовательский интерфейс и логику отображения данных, в то время как сервер выполняет бизнес-логику и хранит данные. Данная архитектура проста и применяется в небольших системах, но может иметь ограничения в масштабируемости и управлении

данными. На рисунке 4 изображена схема работы приложения с двухзвенной КСА.

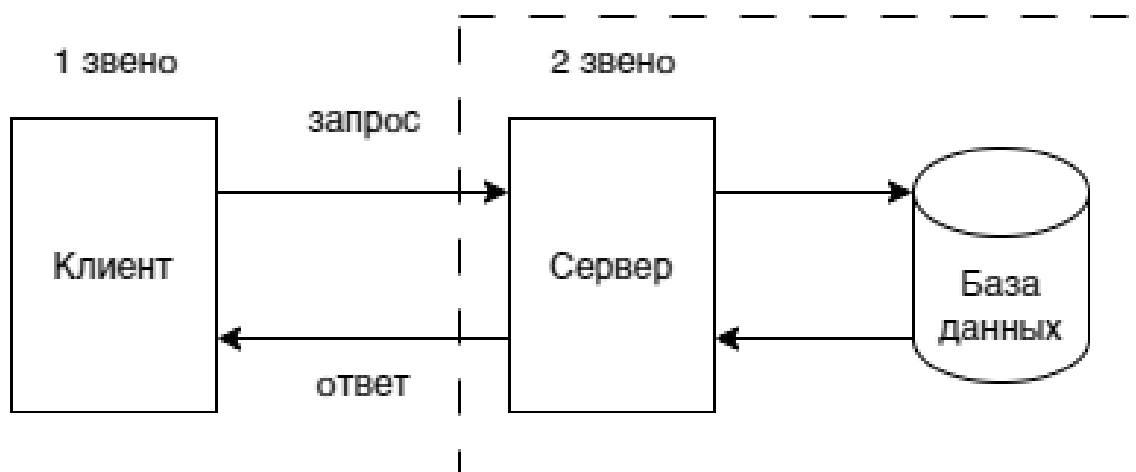


Рисунок 4 – Схема работы приложения с двухзвенной КСА

Трехзвенная архитектура – трехзвенная архитектура разбивает клиент-серверное приложение на три основных компонента или уровня [11]. Первый уровень – клиентский интерфейс, который обрабатывает взаимодействие с пользователем.

Второй уровень – сервер приложений (бизнес-логика), который обрабатывает запросы клиентов и взаимодействует с базой данных. Третий уровень – сервер баз данных, который отвечает за хранение и управление данными. Эта архитектура позволяет более гибко масштабировать и управлять приложением, а также легче разделять обязанности между уровнями. На рисунке 5 изображена схема работы приложения с трехзвенной КСА.

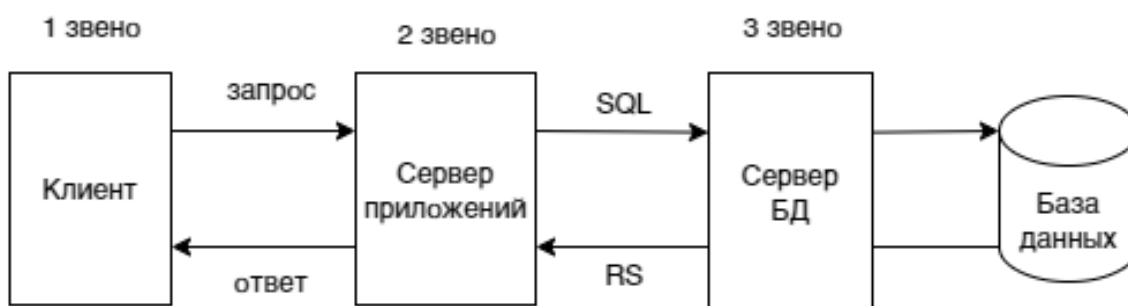


Рисунок 5 – Схема работы приложения с трехзвенной КСА

Для разрабатываемого приложения наиболее подходящим вариантом является двухзвенная КСА, потому что данная архитектура:

- обеспечивает прямое и быстрое взаимодействие между клиентом и сервером без лишних промежуточных звеньев, что особенно важно для передачи данных в режиме реального времени;

- обеспечивает быстрый отклик системы на запросы клиента, что критически важно для реализации передачи данных в реальном времени;

- позволяет легко управлять правами доступа. Сервер может контролировать, какие данные могут быть доступны пользователям, и обеспечивать шифрование для защиты конфиденциальных данных.

2.2 Структурная схема системы

Сущность структурного подхода к проектированию системы заключается в ее декомпозиции на автоматизируемые функции: система разбивается на функциональные подсистемы, которые в свою очередь делятся на подфункции, подразделяемые на задачи и так далее. Процесс разбиения продолжается вплоть до конкретных процедур. При этом автоматизируемая система сохраняет целостное представление, в котором все составляющие компоненты взаимоувязаны. При разработке системы «снизу-вверх» от отдельных задач ко всей системе целостность теряется, возникают проблемы при информационной стыковке отдельных компонентов [12].

Все наиболее распространенные методологии структурного подхода базируются на ряде общих принципов. В качестве двух базовых принципов используются следующие:

- принцип «разделяй и властвуй» – принцип решения сложных проблем путем их разбиения на множество меньших независимых задач, легких для понимания и решения;

- принцип иерархического упорядочивания – принцип организации составных частей проблемы в иерархические древовидные структуры с добавлением новых деталей на каждом уровне.

На рисунке 6 приведена структурная схема разрабатываемой системы.

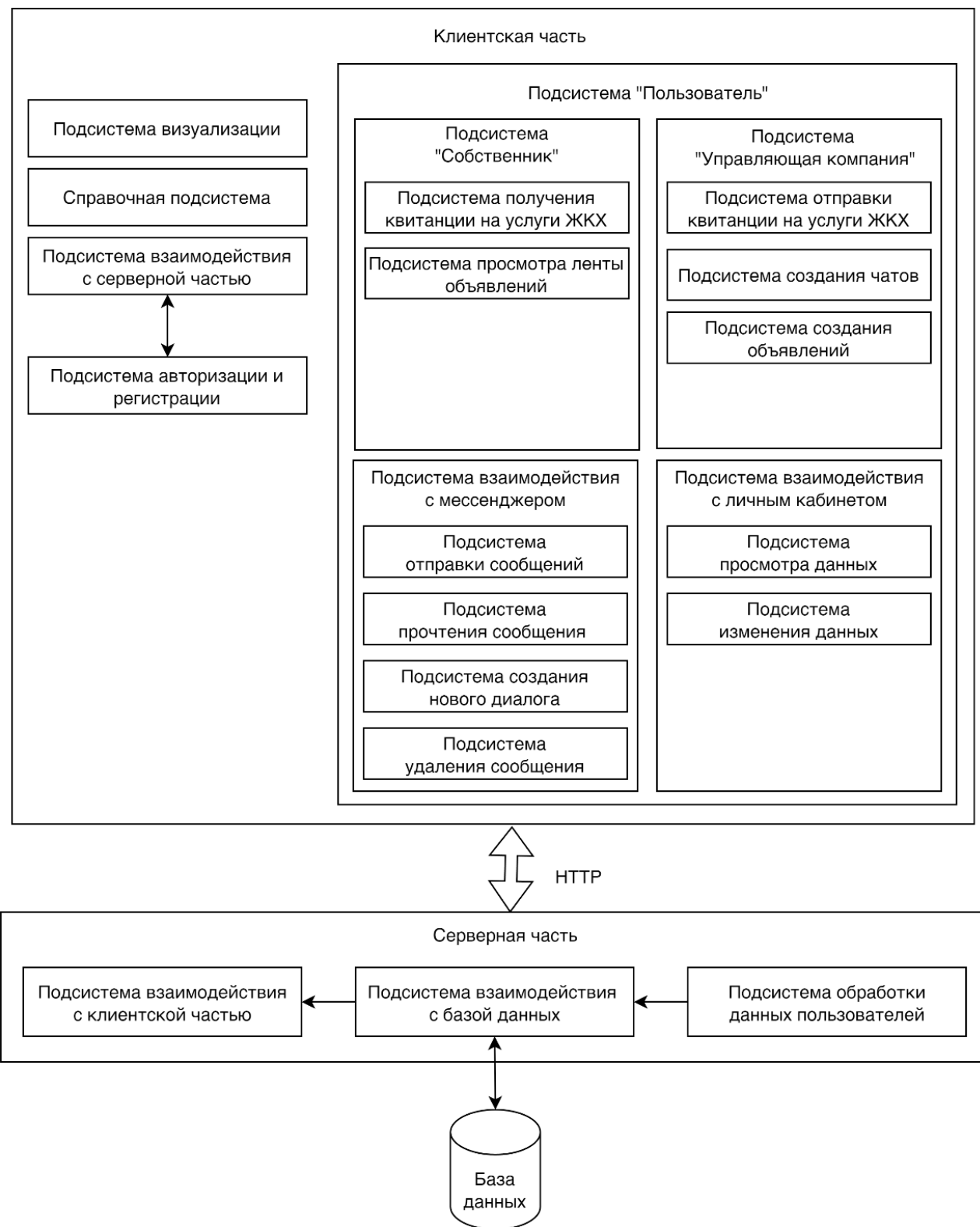


Рисунок 6 – Структурная схема системы

Структурную схему можно подразделить на клиентскую часть, серверную часть и базу данных. В состав клиентской части входят:

- 1) подсистема «Пользователь», в состав которой входят:
 - а) подсистема «Собственник», в состав которой входят»:

- подсистема получения квитанции на услуги ЖКХ, которая отвечает за получение и доступ к квитанциям на услуги ЖКХ;

- подсистема просмотра ленты объявлений, которая отвечает за доступ к ленте объявлений;

б) подсистема «Управляющая компания», в которую включены:

- подсистема отправки квитанций на услуги, которая отвечает за возможность отправить квитанции на услуги ЖКХ собственникам;

- подсистема создания чата, которая отвечает за возможность создания чатов с 3-мя и более участниками;

- подсистема создания объявлений, которая отвечает за возможность создания объявления;

в) подсистема взаимодействия с мессенджером, в которую включены:

- подсистема отправки сообщений; которая отвечает за возможность отправки сообщения в чат;

- подсистема прочтения сообщения, которая отвечает за возможность прочтения сообщения в чате;

- подсистема создания нового диалога, которая отвечает за возможность создания нового диалога в мессенджере;

- подсистема создания удаления сообщения, которая отвечает за возможность удаления сообщения пользователя в диалоге;

г) Подсистема взаимодействия с личным кабинетом, в которую включены:

- подсистема просмотра данных; которая отвечает за возможность просмотра данных пользователя в личном кабинете;

- подсистема изменения данных, которая отвечает за возможность изменения данных пользователя в личном кабинете;
- 2) справочная подсистема, которая отвечает за выдачу справки о системе и ее разработчиках;
- 3) подсистема визуализации, которая отвечает за визуализацию всех составляющих приложения;
- 4) подсистема авторизации и регистрации, которая отвечает за возможность пользователя авторизоваться или зарегистрироваться в системе;
- 5) подсистема взаимодействия с серверной частью, которая обеспечивает взаимодействие клиентской части с серверной частью.

В состав серверной части входят следующие подсистемы:

- 1) подсистема обработки действий пользователя, отвечающая за принятие данных и запросов, приходящих из клиентской части;
- 2) подсистема взаимодействия с базой данных, отвечающая за работу с базой данных в зависимости от запросов пользователя;
- 3) подсистема взаимодействия с клиентской частью, отвечающая за обеспечение взаимодействия серверной части с клиентской.

2.3 Разработка прототипа интерфейса пользователя системы

Интерфейс – это средство взаимодействия пользователя с программным обеспечением [13].

Прототипирование помогает выявить потенциальные проблемы или недостатки интерфейса, что дает возможность внести корректировки на ранних стадиях разработки [14].

Для проектирования форм приложения был использован графический редактор Figma. В нём можно создать прототипы сайта, интерфейсы приложения и обсуждать правки с коллегами в реальном времени [15].

Основными требованиями к интерфейсу являются:

– функциональность – набор возможностей, операций и действий, предоставляемых пользователю или другим системам через данный интерфейс;

– понятность – интуитивно понятный интерфейс.

На рисунке 7 представлен прототип главной страницы веб-приложения для неавторизованного пользователя.

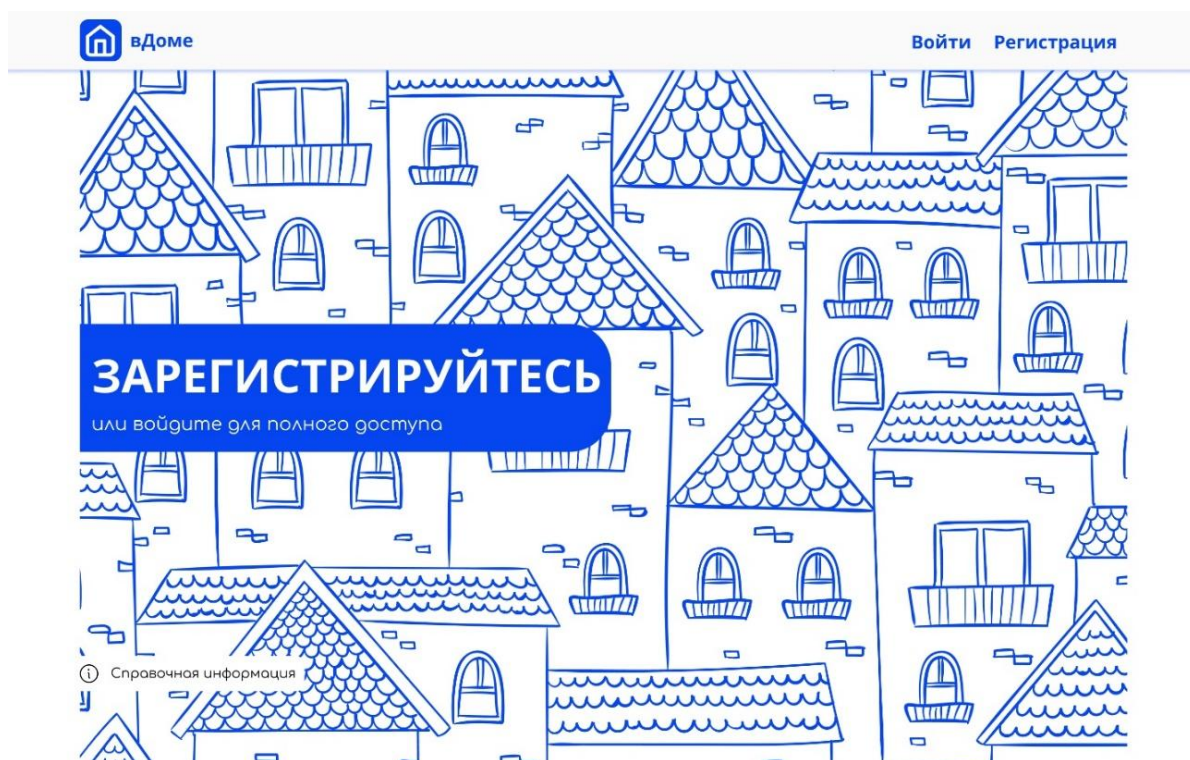
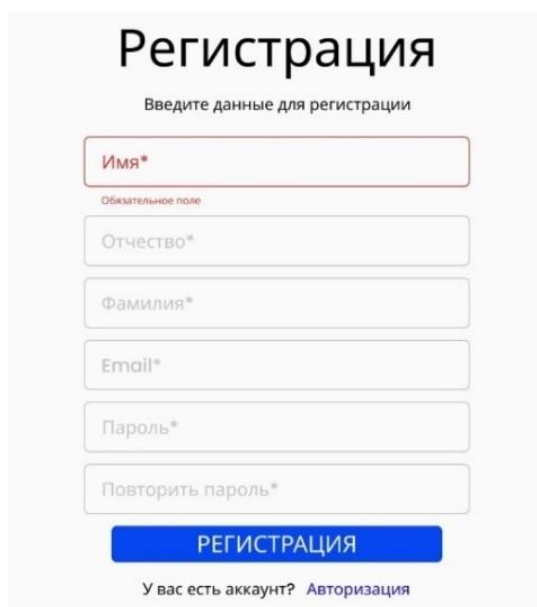


Рисунок 7 – Прототип главной страницы неавторизованного пользователя

На ней размещается шапка сайта с кнопками «Войти» и «Регистрация», при нажатии которых система открывает соответствующие формы. С этой страницы пользователь может перейти к просмотру справочной информации, зарегистрироваться или авторизоваться.

По нажатию на кнопку «Регистрация» открывается форма регистрации. Форма содержит 6 текстовых полей: «Имя*», «Отчество*», «Фамилия*», «Email*», «Пароль*», «Повторите пароль*», – и кнопку «Регистрация», с помощью которой происходит валидация пользователя в системе, ссылку «Авторизация» для перехода на другую страницу с целью повторного входа. В случае некорректного ввода данных пользователь увидит сообщение об

ошибке, и поле подсветится красным. На рисунке 8 приведён её разработанный прототип.



Регистрация

Введите данные для регистрации

Имя*

Обязательное поле

Отчество*

Фамилия*

Email*

Пароль*

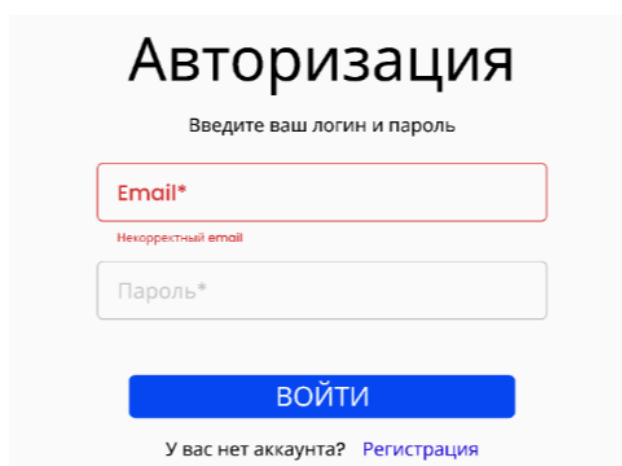
Повторить пароль*

РЕГИСТРАЦИЯ

У вас есть аккаунт? [Авторизация](#)

Рисунок 8 – Прототип формы регистрации

По нажатию на кнопку «Войти» открывается форма авторизации. Форма содержит 2 текстовых поля: «Email*», «Пароль*», – и кнопку «Войти», с помощью которой происходит валидация пользователя в системе, ссылку «Регистрация» для перехода на другую страницу с целью создания учетной записи. В случае некорректного ввода данных пользователь увидит сообщение об ошибке, и поле подсветится красным. На рисунке 9 приведён её разработанный прототип.



Авторизация

Введите ваш логин и пароль

Email*

Некорректный email

Пароль*

ВОЙТИ

У вас нет аккаунта? [Регистрация](#)

Рисунок 9 – Прототип формы авторизации

После успешной регистрации или авторизации пользователь попадает на главную страницу авторизованного пользователя веб-приложения. С этой страницы пользователь может перейти к просмотру справочной информации,

к страницам «Мессенджер», «Личный кабинет», «Квитанции», «Справочная информация», а также выйти из системы. На рисунке 10 приведён её разработанный прототип.

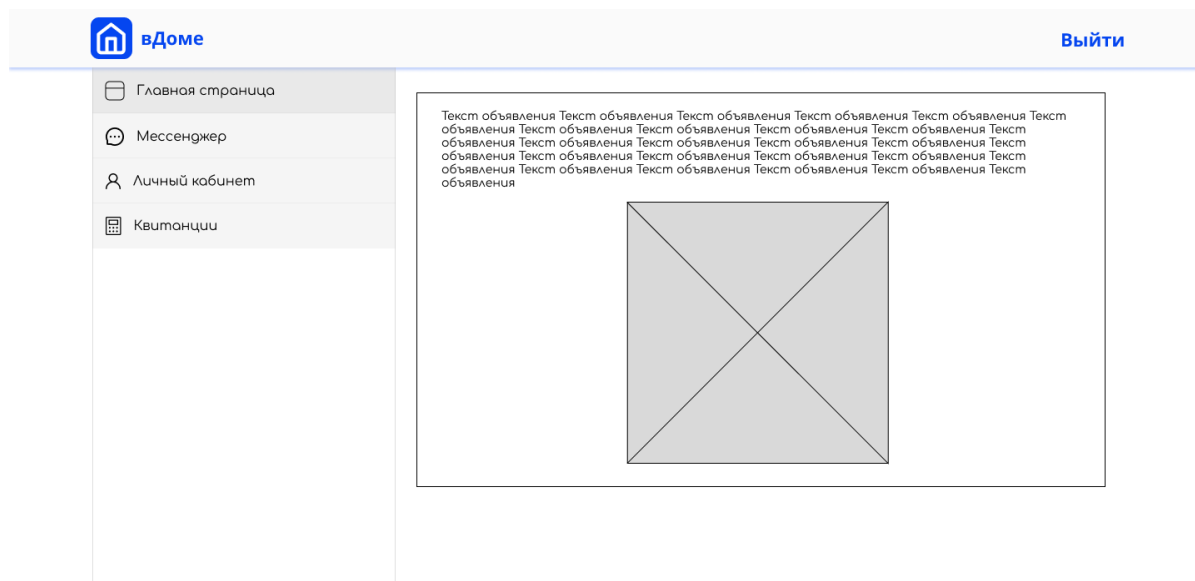


Рисунок 10 – Прототип главной страницы авторизованного пользователя

По нажатию на кнопку «Справочная информация» открывается страница со справочной информацией о разработанной системе и разработчике. С этой страницы пользователь может перейти к страницам, «Главная», «Мессенджер», «Личный кабинет», «Квитанции», а также выйти из системы. На рисунке 11 приведён её разработанный прототип.

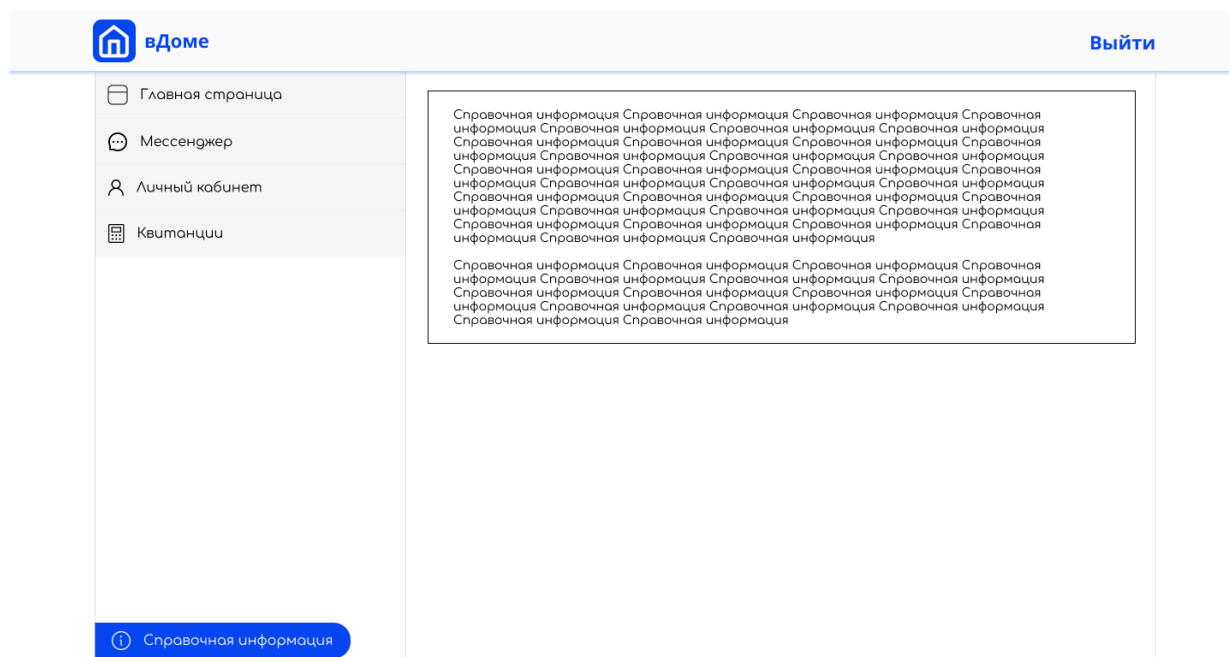


Рисунок 11 – Прототип страницы со справочной информации

По нажатию на кнопку «Мессенджер» открывается страница мессенджера. В верхней части веб-страницы находится шапка веб-приложения с доступными кнопками «Выйти» и «Логотип», по нажатию на который пользователь перейдет на главную страницу. С левой стороны веб-страницы расположен список доступных пользователю чатов, кнопки «Создать чат» и «Поиск диалога». С правой стороны веб-страницы находится текущий диалог с полем для ввода нового сообщения снизу и с фотографией, названием чата и кнопкой «Другое» сверху. С этой страницы пользователь может перейти к странице «Главная», а также выйти из системы. На рисунке 12 приведён её разработанный прототип.

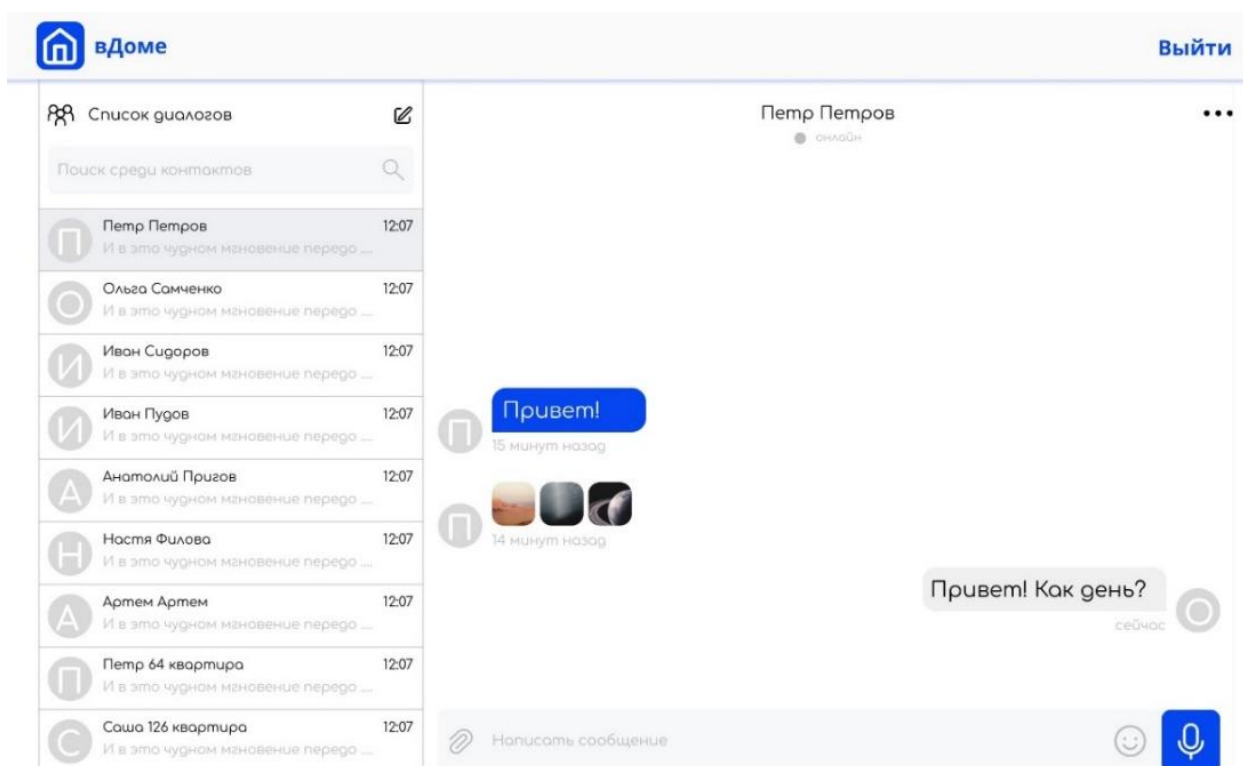


Рисунок 12 – Прототип страницы мессенджера

По нажатию на кнопку «Личный кабинет» открывается страница личного кабинета пользователя. На данной странице пользователь изменить фотографию и личные данные: номер телефона и совместно проживающих граждан. С этой страницы пользователь может перейти к страницам «Главная», «Мессенджер», «Квитанции», «Справочная информация» и выйти из системы. На рисунке 13 представлен ее разработанный прототип.

Рисунок 13 – Прототип экранной формы личного кабинета

По нажатию на кнопку «Квитанции» открывается страница со списком доступных квитанций. С этой страницы пользователь может перейти к страницам «Мессенджер», «Личный кабинет», «Справочная информация» а также выйти из системы. На рисунке 14 представлен ее разработанный прототип.

Дата	Статус
За сентябрь 2023 квартира 45	не оплачено
За август 2023 квартира 45	оплачено
За июль 2023 квартира 45	оплачено
За июнь 2023 квартира 45	оплачено

Рисунок 14 – Прототип экранной формы квитанции

2.4 Разработка информационно-логического проекта системы

2.4.1 Язык UML

Одним из инструментов разработки спецификации системы является язык UML.

Унифицированный язык моделирования (Unified Modeling Language – UML) – это стандартный инструмент для разработки «чертежей» программного обеспечения. Его можно использовать для визуализации, спецификации, конструирования и документирования артефактов программных систем. UML подходит для моделирования любых систем – от информационных систем масштаба предприятия до распределенных веб-приложений и даже встроенных систем реального времени [16].

Важная особенность UML – этот язык поддерживает объектно-ориентированный подход, где все сущности представлены как объекты с определенными свойствами и методами. В диаграммах UML легко изобразить объекты, связи между ними, наследование и возможности передачи данных от одного объекта к другому.

2.4.2 Диаграмма вариантов использования

Диаграмма вариантов использования представляет собой наиболее общую концептуальную модель сложной системы, которая является исходной для построения всех остальных диаграмм. На ней изображаются отношения между актерами и вариантами использования [17].

Актер (actor) – согласованное множество ролей, которые играют внешние сущности по отношению к вариантам использования при взаимодействии с ними (это может быть любой объект, субъект или система, взаимодействующая с моделируемой бизнес-системой извне).

Вариант использования – внешняя спецификация последовательности действий, которые система или другая сущность могут выполнять в процессе взаимодействия с актерами (он определяет набор действий, совершаемый системой при диалоге с актером) [18].

На рисунке 15 представлена общая часть диаграммы вариантов использования, которая отображает действия для неавторизованного пользователя, который может пройти авторизацию или регистрацию, а также просмотреть справочную информацию.

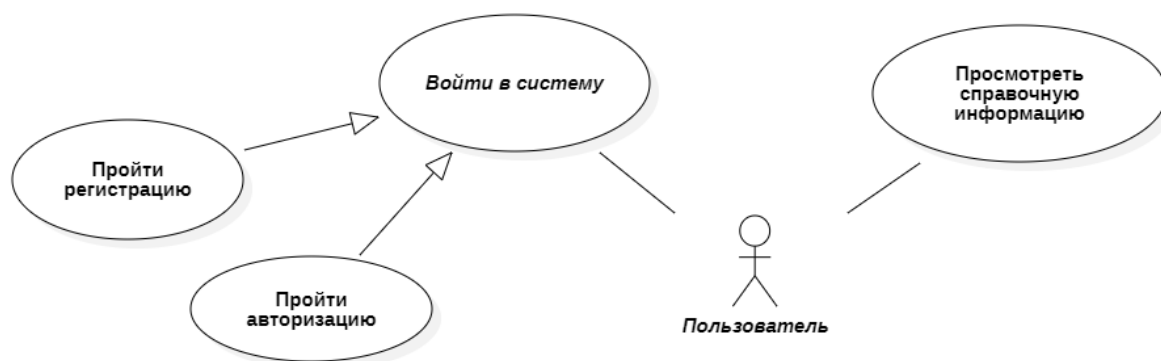


Рисунок 15 – Диаграмма вариантов использования (общая часть)

На рисунке 16 представлена диаграмма вариантов использования для ролей пользователя «Собственник» и «Управляющая компания».

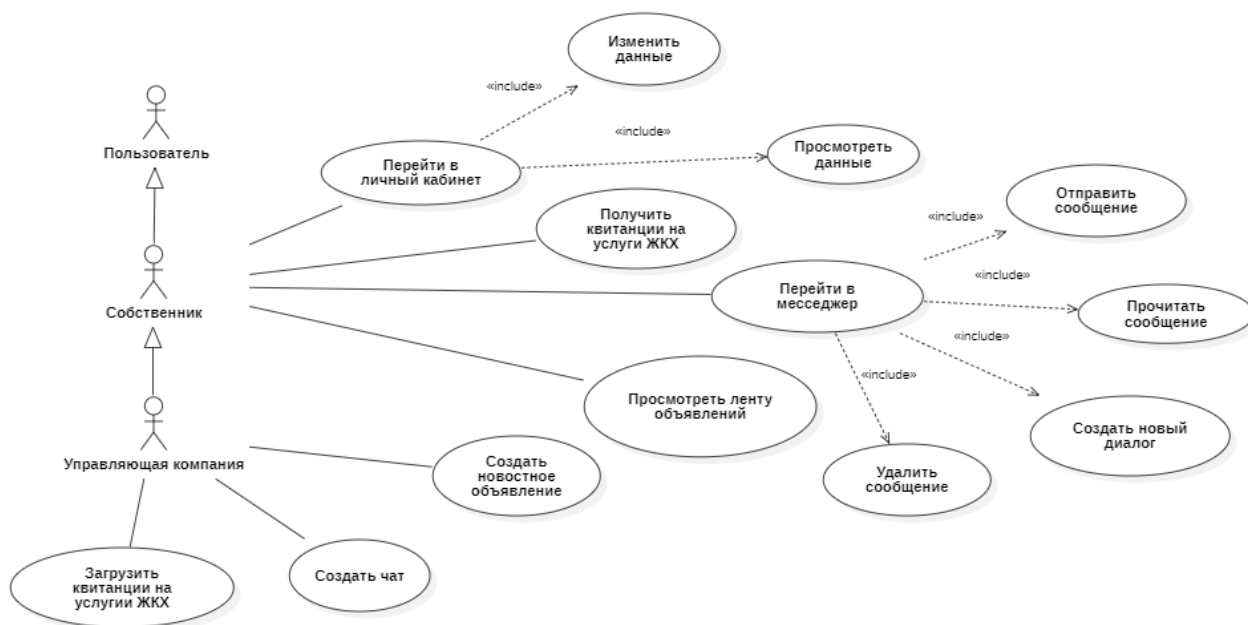


Рисунок 16 – Диаграмма вариантов использования (режимы «Собственник» и «Управляющая компания»)

В роли «Собственник» пользователю доступны функции получить квитанцию на услуги ЖКХ, перейти в личный кабинет, перейти в мессенджер, просмотреть ленту объявлений.

Режим «Управляющая компания» позволяет пользователю загрузить квитанции на услуги ЖКХ, создать чат, создать новостное объявление.

2.4.3 Сценарий

Сценарий (scenario) – определенная последовательность действий, которая описывает действия актеров и поведение моделируемой системы в форме обычного текста [16].

В контексте языка UML сценарий используется для дополнительной иллюстрации взаимодействия актеров и вариантов использования.

Рассмотрим сценарий для варианта использования: «Отправить сообщение».

Краткое описание. Позволяет пользователю в текущий чат отправить сообщение, которое он напечатал. Включается в вариант использования «Перейти в мессенджер».

Актант. Пользователь.

Предусловия: Компьютер включен, авторизация на веб-сайте успешно пройдена. На экране интерфейс страницы «Мессенджер».

1 В верхней части веб-страницы находится шапка веб-приложения с доступными кнопками «Выйти» и «Логотип» для перехода к главной странице. С левой стороны веб-страницы расположен список доступных пользователю чатов, кнопки  «Создать чат» и  «Поиск диалога». С правой стороны веб-страницы находится текущий диалог с полем для ввода нового сообщения снизу, названием чата и кнопкой  «Другое» сверху.

2 Пользователь нажимает левой кнопкой мыши (ЛКМ) кнопку «Отправить сообщение».

A1: Пользователь нажимает ЛКМ на кнопку  «Создать чат».

A2: Пользователь нажимает ЛКМ на кнопку  «Поиск диалога».

A3: Пользователь нажимает ЛКМ на кнопку «Выйти».

A4: Пользователь нажимает ЛКМ на чат с другим пользователем.

A5: Пользователь нажимает ЛКМ на кнопку «Логотип» для перехода к главной странице.

3 Система отправляет сообщение в текущий чат. Вариант использования завершается успешно.

Альтернативы:

A1: Пользователь нажимает ЛКМ на кнопку  «Создать чат».

A1.1: Переход к варианту использования «Создать новый чат».

A2: Пользователь нажимает ЛКМ на кнопку  «Поиск диалога».

A2.1: Пользователь вводит название диалога.

A2.2: Переход к найденному диалогу.

A2.3: Переход к варианту использования «Прочитать сообщение».

A3: Пользователь нажимает ЛКМ на кнопку «Выйти».

A3.1: Переход на главную страницу системы.

A3.2: Переход к варианту использования «Войти в систему».

A4: Пользователь нажимает ЛКМ на чат с другим пользователем.

A4.1: Пользователь переходит к выбранному чату.

A4.2: Переход к варианту использования «Прочитать сообщение».

A5: Пользователь нажимает ЛКМ на кнопку «Логотип» для перехода к главной странице.

A5.1: Переход к варианту использования «Перейти к новостной ленте».

Постусловия. При успешном завершении на экране – страница веб-приложения с отправленным сообщением в чат.

2.4.4 Диаграмма состояний

Для моделирования поведения на логическом уровне в языке UML могут использоваться сразу несколько канонических диаграмм: состояний, деятельности и последовательности, каждая из которых фиксирует внимание на отдельном аспекте функционирования системы [19].

В отличие от других диаграмм, диаграмма состояний описывает процесс изменения состояний только одного экземпляра определенного

класса, т. е. моделирует все возможные изменения в состоянии конкретного объекта.

Главное предназначение этой диаграммы – описать возможные последовательности состояний и переходов, которые в совокупности характеризуют поведение элемента модели в течение его жизненного цикла.

При запуске система отображает главную страницу веб-приложения. Пользователь может просмотреть главную страницу неавторизованного пользователя, перейти на страницу со справочной информацией, войти в существующую учетную запись или зарегистрироваться. На рисунке 17 приведена общая часть диаграммы состояний системы.

После входа в систему в качестве собственника пользователю становятся доступными страницы: главная страница авторизованного пользователя, личный кабинет, мессенджер, квитанции. Для завершения работы в системе, пользователь может нажать на «X». На рисунке 18 приведена диаграмма состояний в режиме «Собственник».

После входа в систему в роли управляющей компании пользователю становятся доступными страницы: главная страница авторизованного пользователя, личный кабинет, мессенджер, квитанции. Для завершения работы в системе, пользователь может нажать на «X». На рисунке 19 приведена диаграмма состояний в режиме «Управляющая компания».

2.4.5 Диаграмма деятельности

Диаграмма деятельности (активностей – Activity Diagram) применяется для моделирования динамических аспектов поведения системы. Это частный случай диаграммы состояний, она позволяет реализовать особенности процедурного и синхронного управления, обусловленного завершением внутренних действий и деятельности [20]. Эта диаграмма представляет собой блок-схему, которая наглядно показывает, как поток управления переходит от одной деятельности к другой.

Пользователь может отправлять текстовые сообщения, выбирать смайлики, добавлять медиафайлы или аудиосообщения. При нажатии на

кнопку "Отправить", система проверяет корректность введённого сообщения и отправляет его, если ввод был произведён успешно.

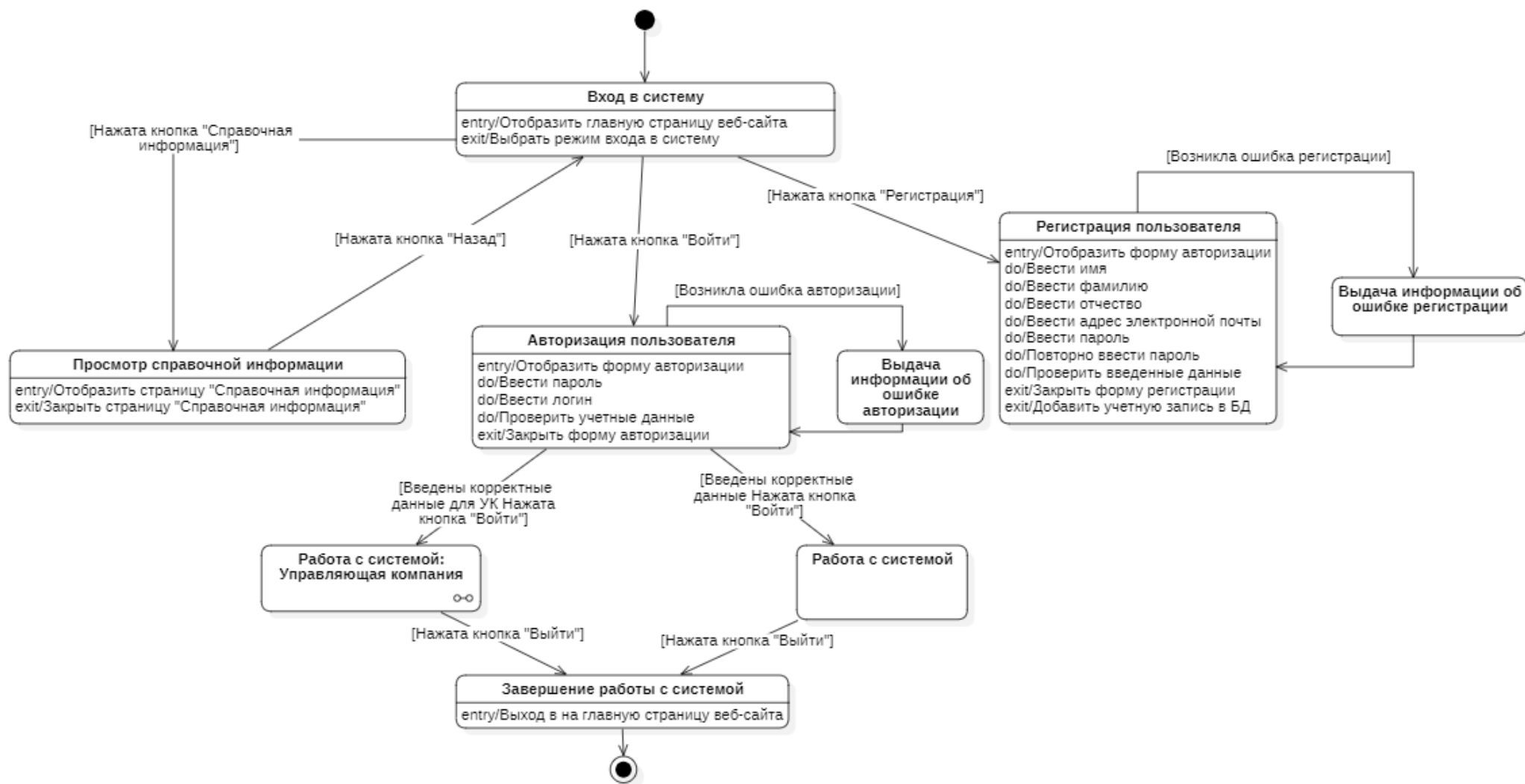


Рисунок 17 – Диаграмма состояний (общая часть)

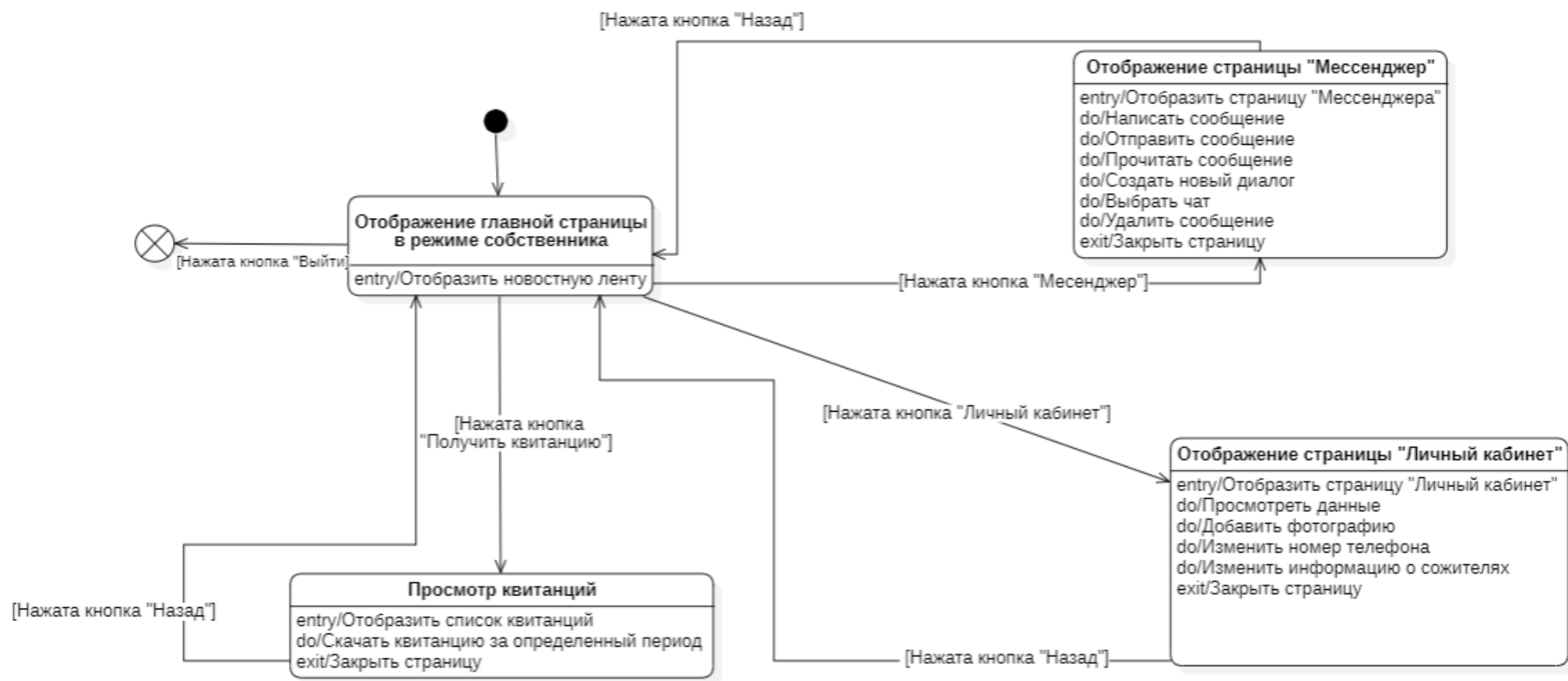


Рисунок 18 – Диаграмма состояний (режим «Собственник»)

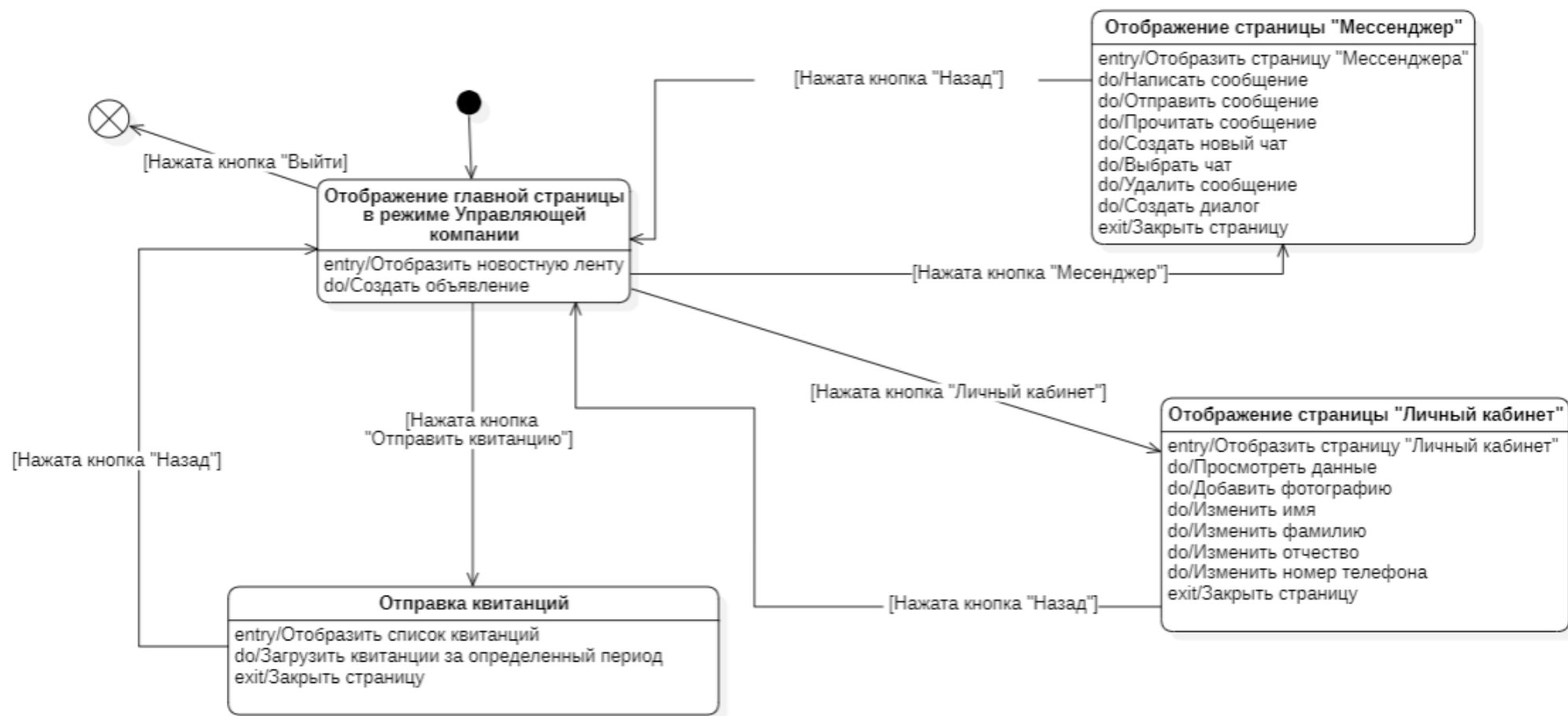


Рисунок 19 – Диаграмма состояний (режим «Управляющая компания»)

На рисунке 20 представлена диаграмма деятельности системы для варианта использования «Отправить сообщение».

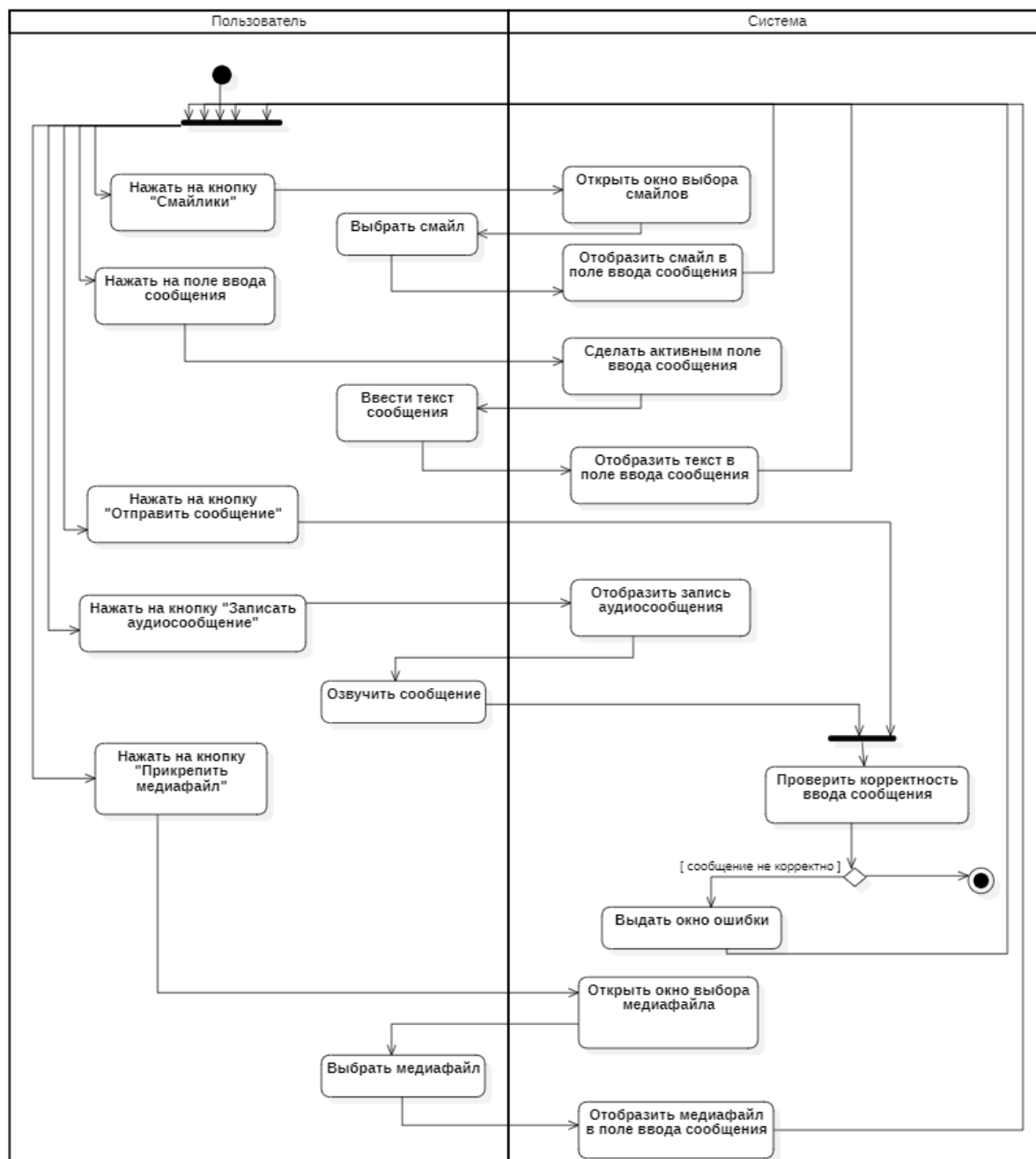


Рисунок 20 – Диаграмма деятельности для варианта использования «Отправить сообщение»

2.4.6 Диаграмма классов

Диаграммы классов – это наиболее часто используемый тип диаграмм, которые создаются при моделировании объектно-ориентированных систем, они показывают набор классов, интерфейсов и коопераций, а также их связи. На практике диаграммы классов применяют для моделирования статического

представления системы, они служат основой для целой группы взаимосвязанных диаграмм – диаграмм компонентов и диаграмм размещения [21].

На рисунке 21 приведена диаграмма классов системы (этап проектирования).

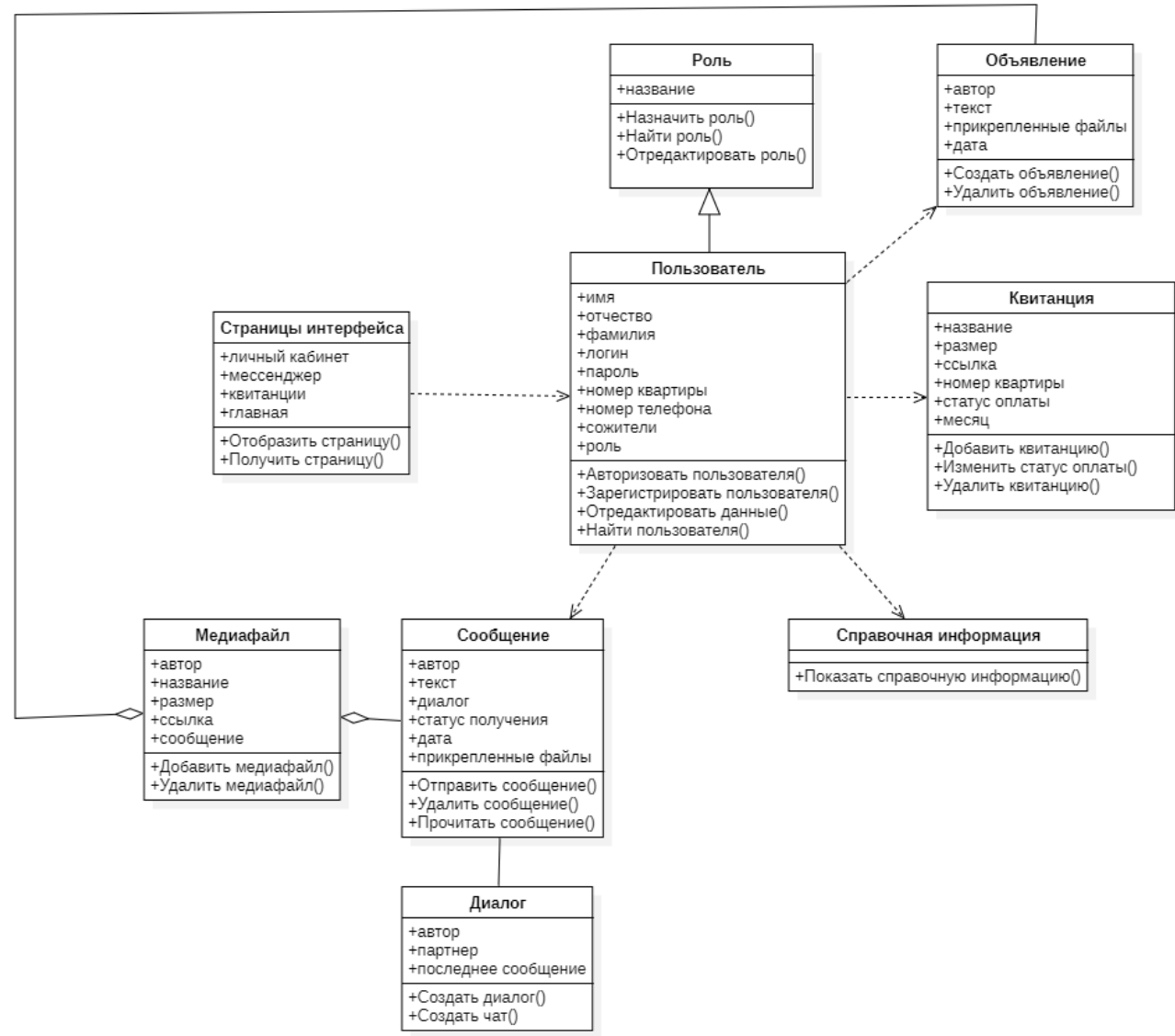


Рисунок 21 – Диаграмма классов системы (этап проектирования)

В таблице 2 приведено описание классов.

Таблица 2 – Описание классов системы

Название класса	Назначение
1	2
Страницы интерфейса	Реализует пользовательский интерфейс для обычного

	пользователя
--	--------------

Продолжение таблицы 2

1	2
Пользователи	Хранит информацию о пользователях и реализует процесс аутентификации, авторизации, регистрации, редактирования, поиска пользователя
Роль	Хранит роли и реализует функции назначения, поиска, редактирования ролей пользователей
Квитанция	Хранит информацию о квитанции и реализует функции добавления, удаления квитанции, а также изменения статуса оплаты
Объявление	Хранит информацию об объявлении и реализует функции добавления и удаления объявления
Справочная информация	Реализует функцию отображения справочной информации
Сообщение	Хранит информацию о сообщении и реализует функции отправки и удаления сообщения
Диалог	Хранит информацию о диалоге и реализует функцию создания диалога
Медиафайл	Хранит информацию о медиафайле и реализует функции добавления и удаления медиафайлов

2.4.7 Логическая модель данных

Целью построения логической модели является получение графического представления логической структуры исследуемой предметной области. Логическая модель предметной области иллюстрирует сущности, а также их взаимоотношения между собой.

Основные требования к содержанию модели [22]:

- логическая модель должна отображать все сущности и связи, значимые для той цели, ради которой мы ее рисуем;
- все объекты модели (и сущности, и связи) должны быть именованы, именование сущностей и связей должно выполняться в терминах предметной области;
- для связей должна быть указана кратность (один – многие);

– для каждой связи должно быть указано направление чтения.

Логическая модель базы данных разрабатываемой системы приведена на рисунке 22. Описание объектов рассматриваемой предметной области, которые хранятся в базе данных, приведено в таблицах 3–10.

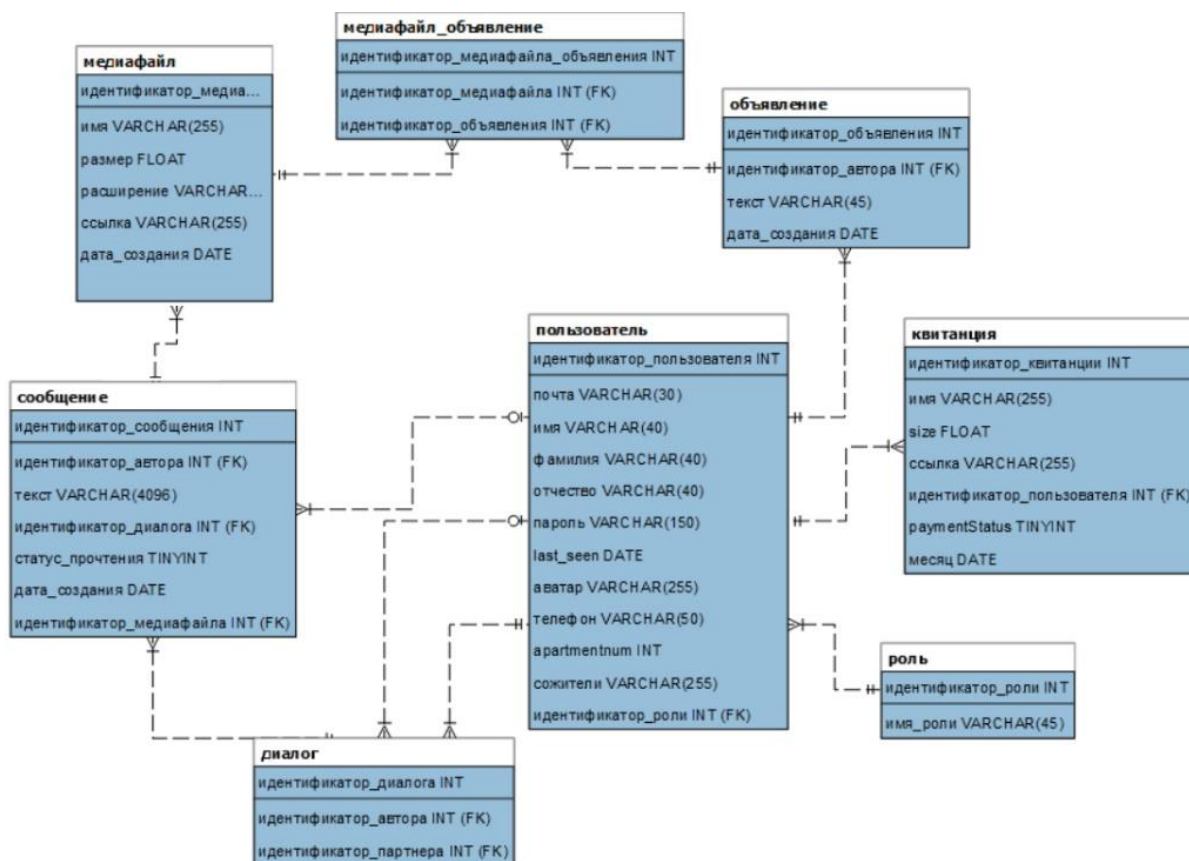


Рисунок 22 – Логическая модель данных

Таблица 3 – Сущность «Пользователь»

Идентификатор	Тип данных	Описание
1	2	3
Уникальный идентификатор пользователя	Целый	Уникальный идентификатор пользователя
Почтовый адрес	Символьный [30]	Почта, выступающая в качестве логина, для авторизации/регистрации пользователя
Имя пользователя	Символьный [40]	Данные пользователя об имени пользователя
Отчество пользователя	Символьный [40]	Данные пользователя об отчестве пользователя

Фамилия пользователя	Символьный [40]	Данные пользователя о фамилии пользователя
-------------------------	-----------------	---

Продолжение таблицы 3

1	2	3
Пароль	Символьный [150]	Закодированный пароль для авторизации/регистрации пользователя
Последнее посещение	Дата	Дата последнего посещения системы пользователем
Аватар	Символьный [255]	Ссылка на изображение аватара пользователя
Телефон	Символьный [50]	Номер телефона пользователя
Номер квартиры	Целый	Номер квартиры пользователя
Сожители	Символьный [255]	Информация о совместно проживающих гражданах
Уникальный идентификатор роли	Целый	Уникальный идентификатор роли пользователя

Таблица 4 – Сущность «Квитанция»

Идентификатор	Тип данных	Описание
Уникальный идентификатор квитанции	Целый	Уникальный идентификатор квитанции
Название	Символьный [45]	Имя, используемое в качестве названия в системе
Размер	Символьный [45]	Объем памяти, который занимает квитанция
Ссылка	Символьный [255]	Ссылка на файл квитанции
Идентификатор пользователя	Целый	Идентификатор собственника, которому выставлен счет
Статус оплаты	Логический	Информация о статусе оплаты квитанции
Месяц	Целый	Информация о месяце, за который выставлен счет в квитанции

Таблица 5 – Сущность «Роль»

Идентификатор	Тип данных	Описание
Уникальный идентификатор роли	Целый	Уникальный идентификатор роли

Название	Символьный [45]	Имя, используемое в качестве названия роли в системе
----------	-----------------	--

Таблица 6 – Сущность «Диалог»

Идентификатор	Тип данных	Описание
Уникальный идентификатор диалога	Целый	Уникальный идентификатор диалога
Уникальный идентификатор автора диалога	Целый	Уникальный идентификатор пользователя (автора диалога)
Уникальный идентификатор партнера	Целый	Уникальный идентификатор пользователя (партнера диалога)
Уникальный идентификатор последнего сообщения	Целый	Уникальный идентификатор сообщения

Таблица 7 – Сущность «Сообщение»

Идентификатор	Тип данных	Описание
Уникальный идентификатор сообщения	Целый	Уникальный идентификатор сообщения
Уникальный идентификатор автора диалога	Целый	Уникальный идентификатор пользователя
Текст	Символьный [4096]	Текст сообщения пользователя
Уникальный идентификатор диалога	Целый	Уникальный идентификатор диалога
Статус получения	Логический	Информация о статусе получения сообщения
Дата отправки	Дата	Дата отправки сообщения
Идентификатор прикрепленного файла	Целый	Уникальный идентификатор прикрепленного файла

Таблица 8 – Сущность «Медиафайл»

Идентификатор	Тип данных	Описание
Уникальный идентификатор медиафайла	Целый	Уникальный идентификатор медиафайла
Имя файла	Символьный [255]	Имя, используемое в качестве названия в системе
Расширение	Символьный [45]	Расширение файла
Размер	Символьный [45]	Объем памяти, который занимает медиафайл
Ссылка	Символьный [255]	Ссылка на медиафайл
Дата загрузки	Дата	Дата загрузки файла

Таблица 9 – Сущность «Объявление»

Идентификатор	Тип данных	Описание
Уникальный идентификатор объявления	Целый	Уникальный идентификатор объявления
Уникальный идентификатор автора объявления	Целый	Уникальный идентификатор пользователя
Текст	Символьный [4096]	Текст объявления
Дата отправки	Дата	Дата отправки сообщения

Таблица 10 – Сущность «Медиафайл_объявления»

Идентификатор	Тип данных	Описание
Уникальный идентификатор медиафайла_объявления	Целый	Уникальный идентификатор медиафайла_объявления
Уникальный идентификатор медиафайла	Целый	Уникальный идентификатор медиафайла
Уникальный идентификатор объявления	Целый	Уникальный идентификатор медиафайла

2.5 Описание комплекса программных средств

Для реализации системы был выбран язык программирования: JavaScript, библиотека для разработки пользовательских интерфейсов: React 18.2.0, кроссплатформенный редактор кода: Visual Studio Code 2024 и СУБД: PostgreSQL 15.

2.5.1 Описание языка программирования

Система будет разрабатываться на языке JavaScript, динамическом скриптовом языке программирования высокого уровня. JavaScript отличается мультипарадигменностью и поддерживает функциональный, императивный и событийно-ориентированный стили.

JavaScript является интерпретируемым языком программирования, который широко используется для создания как frontend-, так и backend-частей веб-сайтов, а также мобильных приложений.

К основным преимуществам JavaScript относятся [23]:

- незаменимость в веб-разработке: JavaScript – основная технология для клиент-серверных моделей и программирования «для интернета»;
- полная интеграция с версткой страниц и серверной частью: язык легко взаимодействует с HTML и CSS, а также с серверными технологиями;
- скорость работы: JavaScript обеспечивает быструю реакцию пользовательского интерфейса;
- производительность: язык оптимизирован для эффективного выполнения кода;
- наличие мощной экосистемы: вокруг JavaScript сформирована обширная инфраструктура, включающая библиотеки, фреймворки и инструменты.

Все эти преимущества делают JavaScript отличным выбором для разработки современных веб-приложений, обеспечивая эффективную, быструю и масштабируемую разработку как для frontend-, так и для backend-частей сайтов.

2.5.2 Описание библиотеки React

Система будет разрабатываться с помощью React библиотека для разработки пользовательских интерфейсов: React 18.2.0. React – это популярный JavaScript-фреймворк, предоставляющий инструменты для создания пользовательских интерфейсов с использованием компонентной архитектуры.

К преимуществам React относят [24]:

- виртуальный DOM: позволяет оптимизировать процесс обновления интерфейса, применяя изменения только к частям, которые действительно изменились, что значительно повышает производительность;
- компонентный подход: способствует повторному использованию кода и созданию модульных приложений, облегчая их поддержку и масштабирование;
- браузерные инструменты для разработчиков: React предоставляет мощные инструменты для отладки и анализа приложений прямо в браузере;
- совместимость с другими технологиями и платформами: React предоставляет гибкость в интеграции с различными технологиями и платформами, что делает его универсальным инструментом для разработки.

Эти преимущества делают React отличным выбором для создания современных, высокопроизводительных и масштабируемых веб-приложений.

2.5.3 Описание среды программирования

Система будет разрабатываться в кроссплатформенном редакторе кода Visual Studio Code (VS Code) – это редактор кода для кроссплатформенной разработки веб- и облачных приложений.

VS Code позволяет разрабатывать как консольные приложения, так и приложения с графическим интерфейсом, в том числе с поддержкой технологии Windows Forms, а также веб-сайты, веб-приложения, веб-службы как в родном, так и в управляемом кодах для всех платформ.

В редакторе присутствуют встроенный отладчик, инструменты для работы с Git и средства рефакторинга, навигации по коду, автодополнения типовых конструкций и контекстной подсказки.

Продукт поддерживает разработку для платформ ASP.NET и Node.js, и считается легковесным решением, которое позволяет обойтись без полной интегрированной среды разработки [25].

Благодаря этому Visual Studio Code отлично подходит для разработки веб-приложения.

2.5.4 Описание СУБД

PostgreSQL – это мощная система объектно-реляционной базы данных с открытым исходным кодом, которая использует и расширяет язык SQL в сочетании с множеством функций, обеспечивающих безопасное хранение и масштабирование самых сложных рабочих нагрузок с данными.

PostgreSQL заслужил прочную репутацию благодаря своей проверенной архитектуре, надежности, целостности данных, надежному набору функций, расширяемости и приверженности сообщества разработчиков ПО с открытым исходным кодом к последовательной разработке эффективных и инновационных решений. PostgreSQL работает во всех основных операционных системах, соответствует требованиям ACID с 2001 года и имеет мощные надстройки, такие как популярный расширитель базы данных геопространственных данных PostGIS [26].

2.5.5 Описание операционной системы

Windows – семейство проприетарных операционных систем (ОС) корпорации Microsoft, ориентированных на применение графического интерфейса при управлении. Изначально Windows была всего лишь графической надстройкой-программой для операционной системы 80-х и 90-х годов MS-DOS.

В настоящее время Windows установлена почти на 90% персональных компьютеров и рабочих станций. По данным компании Net Applications, на апрель 2016 года рыночная доля Windows составила 88,77% [27].

Таким образом, Windows является наиболее распространенной операционной системой, а также обладает удобным пользовательским интерфейсом и поддержкой многозадачности.

3 Реализация системы

3.1 Разработка и описание интерфейса пользователя

При запуске системы открывается главная страница веб-приложения для неавторизованного пользователя. На ней размещается шапка сайта с кнопками «Войти» и «Регистрация», при нажатии которых система открывает соответствующие формы. С этой страницы пользователь может перейти к просмотру справочной информации, зарегистрироваться или авторизоваться. На рисунке 23 представлена главная форма приложения.

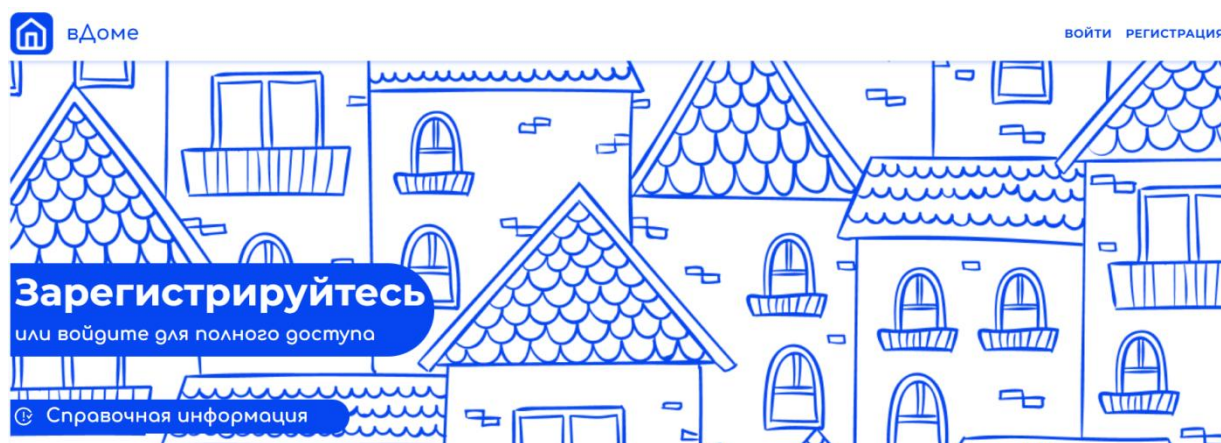


Рисунок 23 – Главная страница неавторизованного пользователя

По нажатию на кнопку «Войти» открывается форма авторизации. Форма содержит 2 текстовых поля: «Email*», «Пароль*», – и кнопку «Войти», с помощью которой происходит валидация пользователя в системе, ссылку «Регистрация» для перехода на другую страницу с целью создания учетной записи. В случае некорректного ввода данных пользователь увидит сообщение об ошибке, и поле подсветится красным. На рисунке 24 представлена форма авторизации пользователя.

Рисунок 24 – Форма авторизации

По нажатию на кнопку «Регистрация» открывается форма регистрации. Форма содержит 6 текстовых полей: «Имя*», «Отчество*», «Фамилия*», «Email*», «Пароль*», «Повторите пароль*», – и кнопку «Регистрация», с помощью которой происходит валидация пользователя в системе, ссылку «Авторизация» для перехода на другую страницу с целью повторного входа. В случае некорректного ввода данных пользователь увидит сообщение об ошибке, и поле подсветится красным. На рисунке 25 представлена форма регистрации пользователя.

Регистрация

Введите данные для регистрации

Имя*
Мария

Отчество*
Александровна

Фамилия*
Агафонова

Email*
maria123@mail.ru

Пароль*
.....

Повторите пароль*
.....

РЕГИСТРАЦИЯ

[У вас есть аккаунт? Авторизация](#)

Рисунок 25 – Форма регистрации

По нажатию на кнопку «Справочная информация» открывается страница со справочной информацией о разработанной системе и разработчике. С этой страницы пользователь может перейти к главной странице, а также выйти из системы. На рисунке 26 представлена страница со справочной информацией.

Справочная информация

Самарский университет
Кафедра программных систем
Выпускная квалификационная работа
Тема проекта: «Разработка веб-приложения для управления многоквартирным домом»
Разработчик: обучающийся группы 6413-020302D Топыркина Алина
Самара, 2024

Назначение системы

Система предназначена на управления многоквартирным домом в режиме реального времени. С помощью системы в роли «Собственник» пользователю доступны функции получить квитанцию на услуги ЖКХ, перейти в личный кабинет, перейти в мессенджер, просмотреть ленту объявлений. В роли «Управляющая компания» система позволяет пользователю загрузить квитанции на услуги ЖКХ, создать чат, создать новостное объявление.

Условие корректной работы системы

Для корректной работы системы необходимо наличие соответствующих программных и аппаратных средств.

1. Требования к техническому обеспечению:

ЭВМ типа IBM PC совместимый

Объем ОЗУ – не менее 1.91 Гб

Рисунок 26 – Страница со справочной информацией

После успешной авторизации пользователь в роли «Собственник» попадает на главную страницу авторизованного пользователя веб-приложения. С этой страницы пользователь может перейти к просмотру справочной информации, к страницам «Мессенджер», «Личный кабинет», «Квитанции», «Справочная информация», а также выйти из системы. На рисунке 27 представлена главную страницу веб-приложения для авторизованного пользователя.

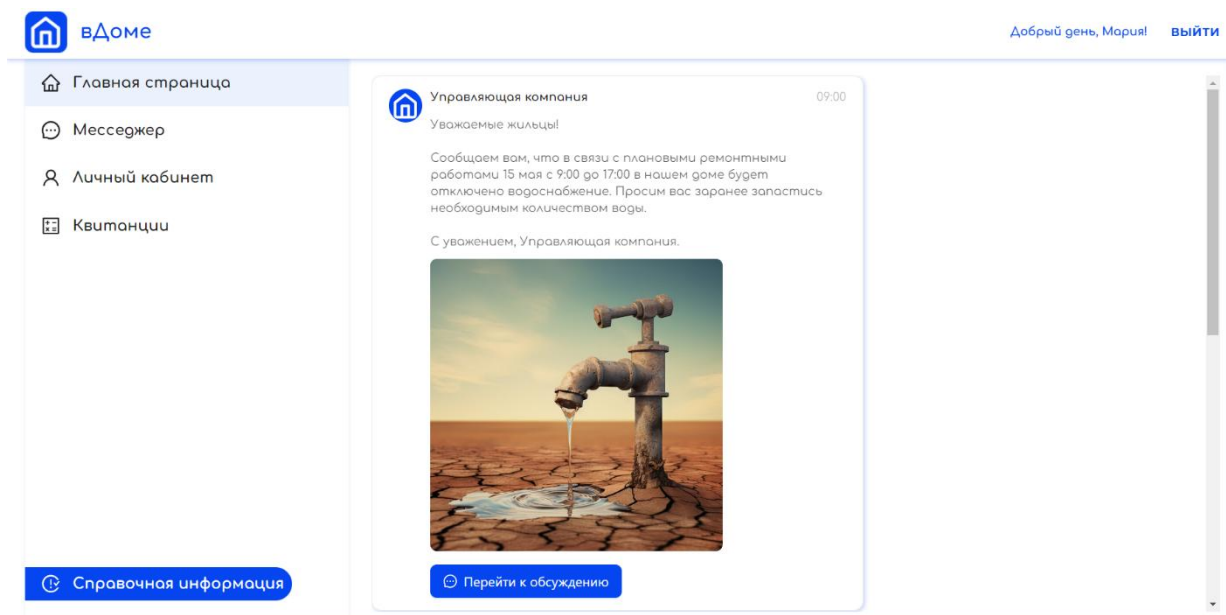


Рисунок 27 – Главная страница авторизованного пользователя

По нажатию на кнопку «Мессенджер» открывается страница с мессенджером веб-приложения. В верхней части веб-страницы находится

шапка веб-приложения с доступными кнопками «Выйти» и «Логотип», по нажатию на который пользователь перейдет на главную страницу. С левой стороны веб-страницы расположен список доступных пользователю чатов, кнопки  «Создать чат» и  «Поиск диалога». С правой стороны веб-страницы находится текущий диалог с текстовым полем для ввода нового сообщения снизу, названием чата сверху. С этой страницы пользователь может перейти главной странице, а также выйти из системы. На рисунке 28 представлена страница с мессенджером.

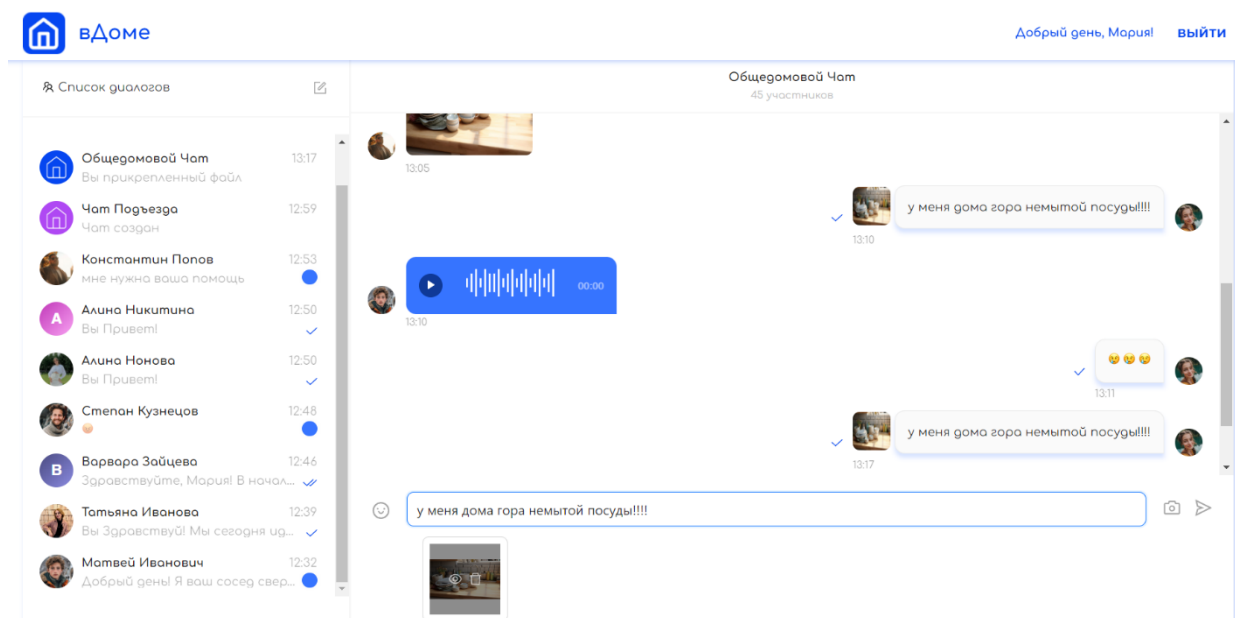


Рисунок 28 – Страница «Мессенджер»

При нажатии на кнопку «Личный кабинет» открывается страница, предоставляющая доступ к персональной информации пользователя. На этой странице пользователь может изменить свою фотографию, а также обновить личные данные, такие как номер телефона и список совместно проживающих граждан. Это обеспечивает актуальность и точность предоставленной информации. С этой страницы пользователь может перейти к страницам «Главная», «Мессенджер», «Квитанции», «Справочная информация» и выйти из системы. На рисунке 29 представлена страница личного кабинета пользователя.

 вДоме

Добрый день, Мария! [Выйти](#)

 Главная страница

 Месседжер

 Личный кабинет

 Квитанции

 Справочная информация



Имя:

Отчество:

Фамилия:

Номер квартиры:

Телефон:

Почта:

С кем живет:

Это поле может изменить только администратор

Сохранить изменения

Рисунок 29 – Страница «Личный кабинет»

По нажатию на кнопку «Квитанции» открывается страница со списком доступных квитанций. С этой страницы пользователь может перейти к страницам «Главная», «Мессенджер», «Личный кабинет», «Справочная информация» а также выйти из системы. На рисунке 30 представлена страница с квитанциями пользователя.

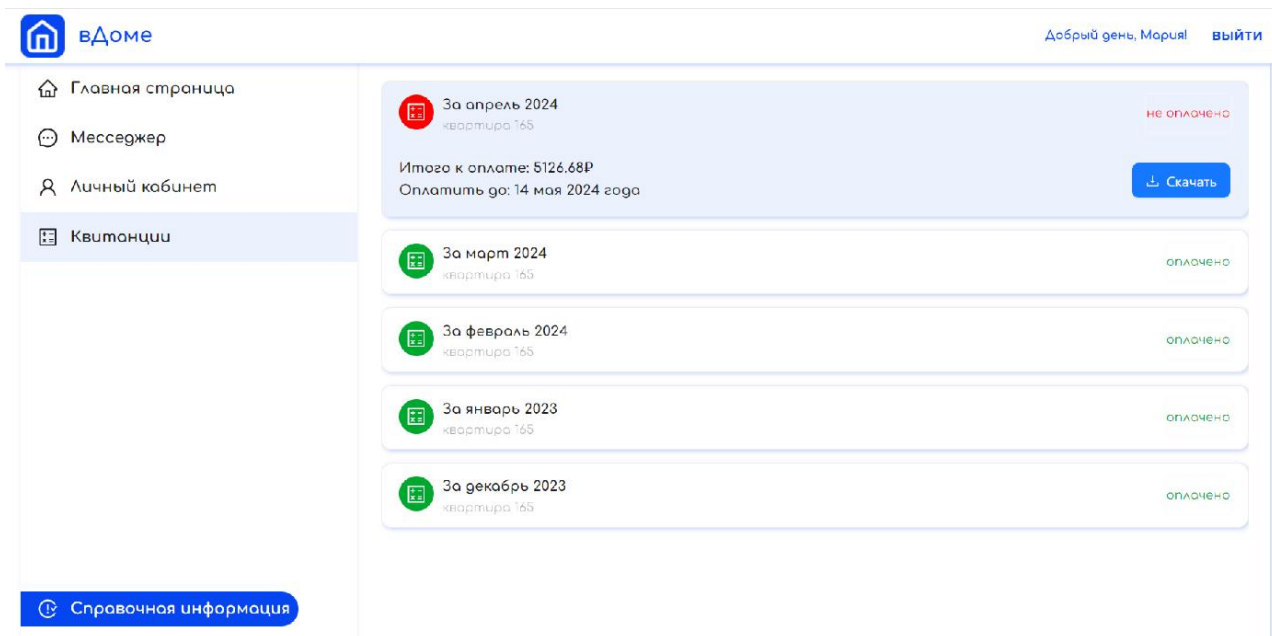


Рисунок 30 – Страница «Квитанции»

После успешной авторизации пользователь в роли «Управляющей компании» также как и «Собственник» попадает на главную страницу авторизованного пользователя веб-приложения, но ему доступен больший функционал. На главной странице авторизованного пользователя «Управляющая компания» может также добавлять новые объявления. На рисунке 31 представлена главная страница авторизованного пользователя «Управляющая компания».

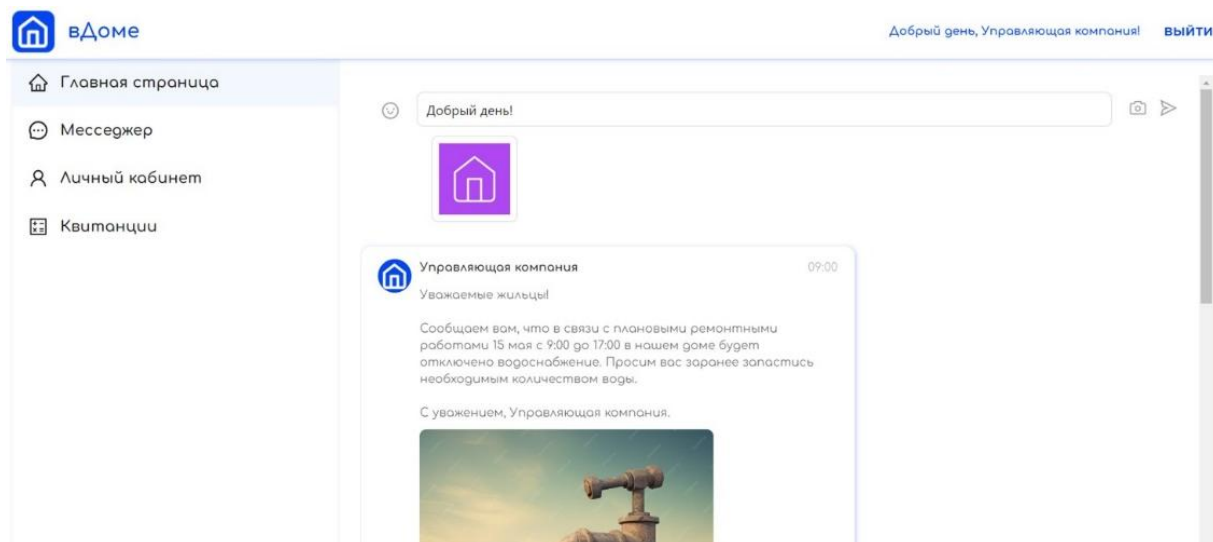


Рисунок 31 – Главная страница авторизованного пользователя

«Управляющая компания»

Пользователь в роли «Управляющая компания» на странице

«Мессенджер» может создавать чаты с 3 и более пользователями. Для этого ему необходимо выбрать собеседников, ввести текстовое сообщение и название чата. На рисунке 32 представлена страница «Мессенджер» авторизованного пользователя «Управляющая компания».

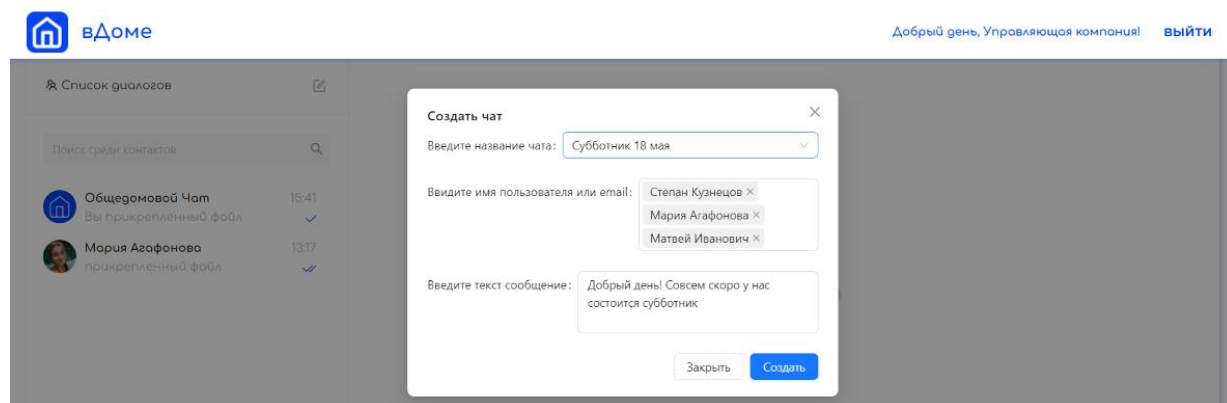


Рисунок 32 – Страница «Мессенджер» авторизованного пользователя «Управляющая компания»

Пользователь в роли «Управляющая компания» на странице «Квитанции» может загружать новые квитанции, изменять статус оплаты. На рисунке 33 представлена страница «Квитанции» авторизованного пользователя «Управляющая компания».

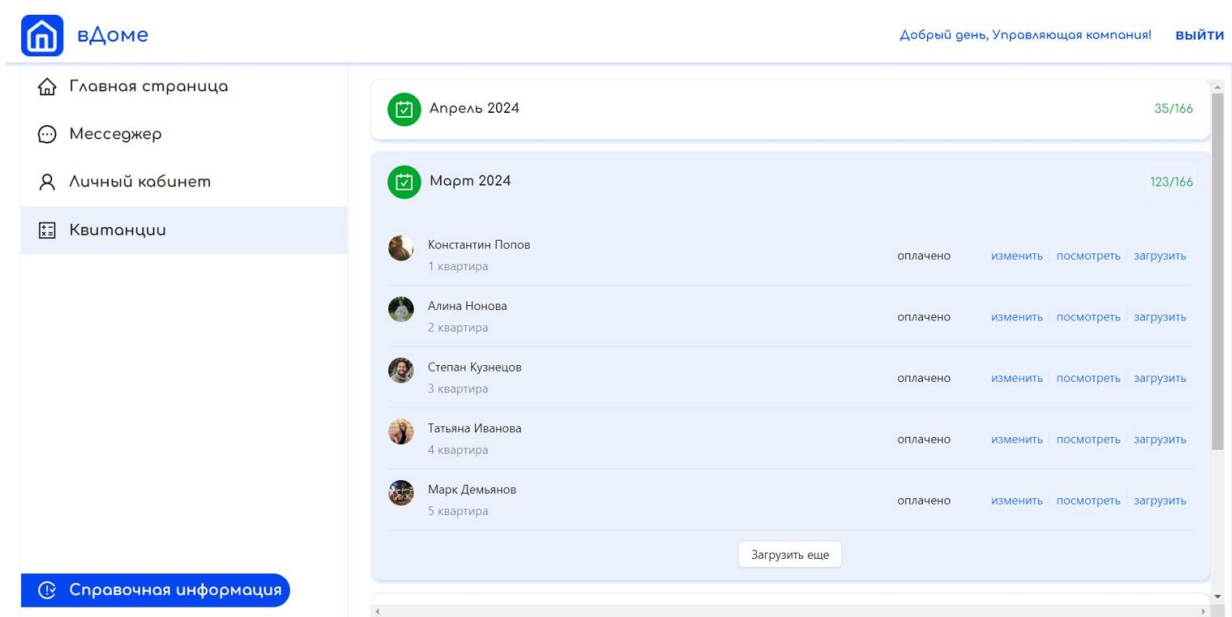


Рисунок 33 – Страница «Квитанции» авторизованного пользователя «Управляющая компания»

3.2 Диаграммы реализации

Диаграммы реализации предназначены для отображения состава компилируемых и выполняемых модулей системы, а также связей между ними. Диаграммы реализации разделяются на два конкретных вида: диаграммы компонентов (component diagrams) и диаграммы развертывания (deployment diagrams) [28].

3.2.1 Диаграмма компонентов

Диаграмма компонентов – диаграмма физического уровня, которая служит для представления программных компонентов и зависимостей между ними [29].

Диаграмма компонентов разрабатывается для визуализации общей структуры исходного кода программной системы, спецификации исполнимого варианта программной системы, обеспечения многократного использования отдельных фрагментов программного кода, представления концептуальной и физической схем баз данных.

Взаимодействие пользователя с системой начинается с главной страницы приложения. Далее пользователь может перейти на страницы авторизации, регистрации, справочной информации. При авторизации идентифицируется роль пользователя и предоставляются дополнительные возможности. Отличительная особенность роли УК – возможность создавать чаты с 3-мя и более участниками, контролировать оплату квитанций на оплату коммунальных услуг. «Собственник» может создавать диалоги только с 2-мя участниками, получать квитанции на оплату коммунальных услуг.

После регистрации пользователь также переходит на страницу аутентификации. Далее он может перейти на страницы мессенджера, личного кабинета, квитанций.

На рисунке 34 приведена диаграмма компонентов, их описание приведено в таблице 11.

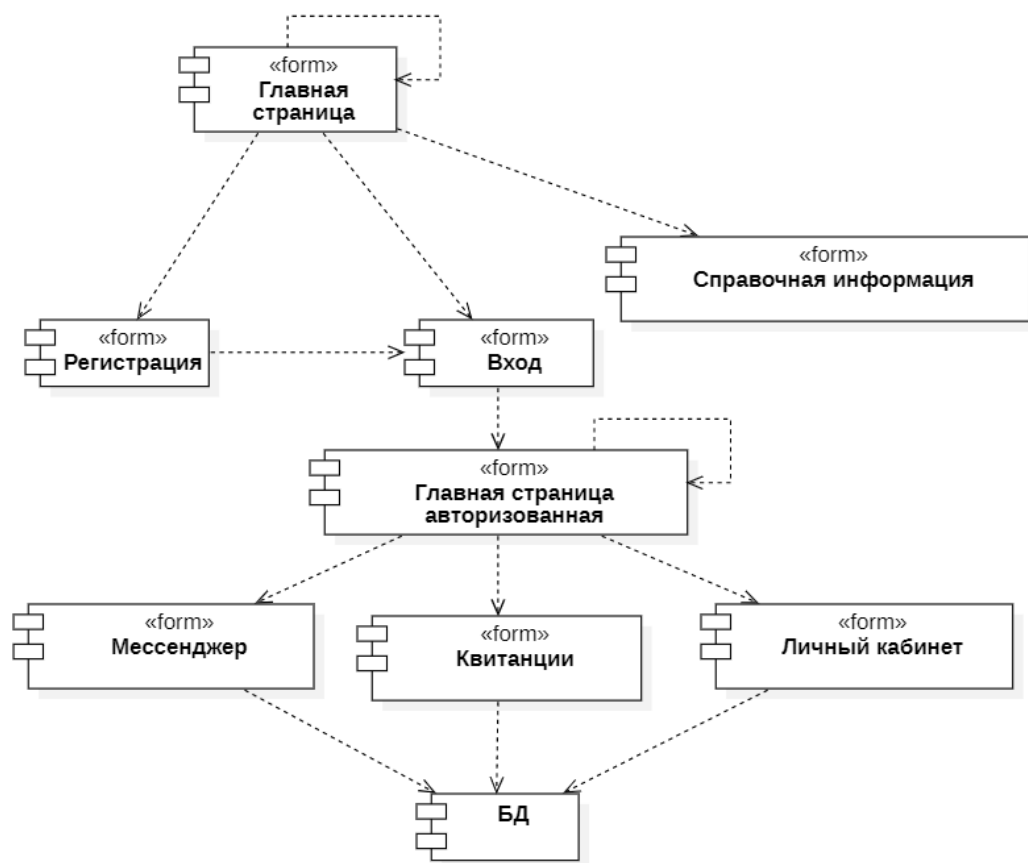


Рисунок 34 – Диаграмма компонентов системы

Таблица 11 – Описание компонентов системы

Название компонента	Назначение
Главная страница	Предназначен для отображения приветственной информации и выбора дальнейших действий
Вход	Предназначен для авторизации пользователя
Регистрация	Предназначен для регистрации пользователя
Справочная информация	Предназначен для отображения справочной информации
Главная страница авторизованная	Предназначен для отображения приветственной информации и расширенного списка возможных дальнейших действий
Мессенджер	Предназначен для коммуникации с другими пользователями в общих чатах многоквартирного дома, чатах по интересам, диалогах
Квитанции	Предназначен для отображения информации о состоянии оплаты квитанций ЖКХ, скачивания квитанции

Личный кабинет	Предназначен для отображения информации и пользователе, а также возможности ее редактирования
----------------	---

3.2.2 Диаграмма развертывания

Диаграммы развертывания показывают конфигурацию исполняемой программной системы, которая включает в себя программные компоненты, процессы и объекты. Состоит же она из узлов и отношений взаимодействия между узлами и компонентами. Узлы могут включать компоненты и объекты.

Узлы могут обозначать компьютеры, человеческие ресурсы или механические устройства. Внутри узлов могут содержаться компоненты что обозначает, что данный компонент или объект существует в рамках данного узла. Узлы изображаются как проекция трехмерного куба [30].

На рисунке 35 представлена диаграмма развертывания. Система разделяется на клиентскую часть и серверную часть. На клиентской части должна быть установлена операционная система и браузер. Серверная часть производит всю основную обработку данных. На ней должна быть установлена операционная система, а также СУБД – PostgreSQL.

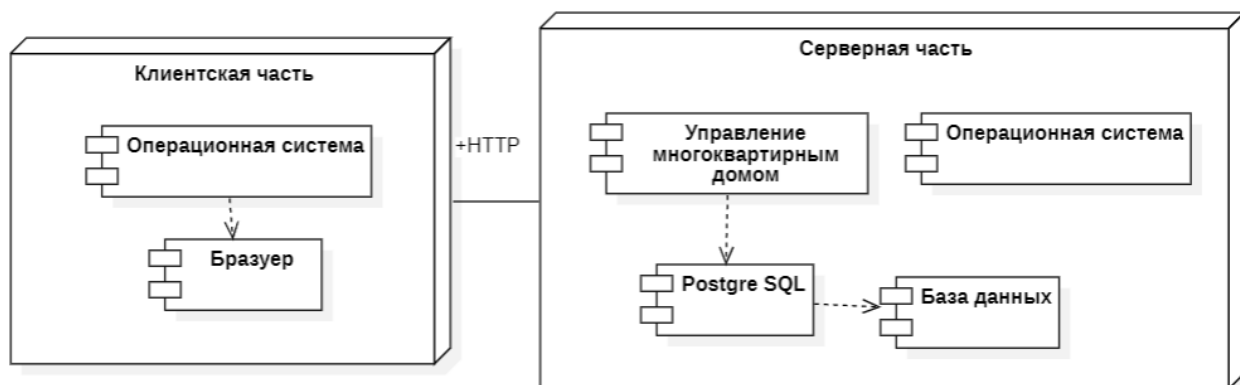


Рисунок 35 – Диаграмма развертывания системы

3.2.3 Диаграмма классов

В соответствии со спецификацией, приведенной в п. 2.4.6, и с учетом выбранного языка программирования (см. п. 2.5.1) разработана диаграмма классов системы (этап реализации), приведенная на рисунке 36.

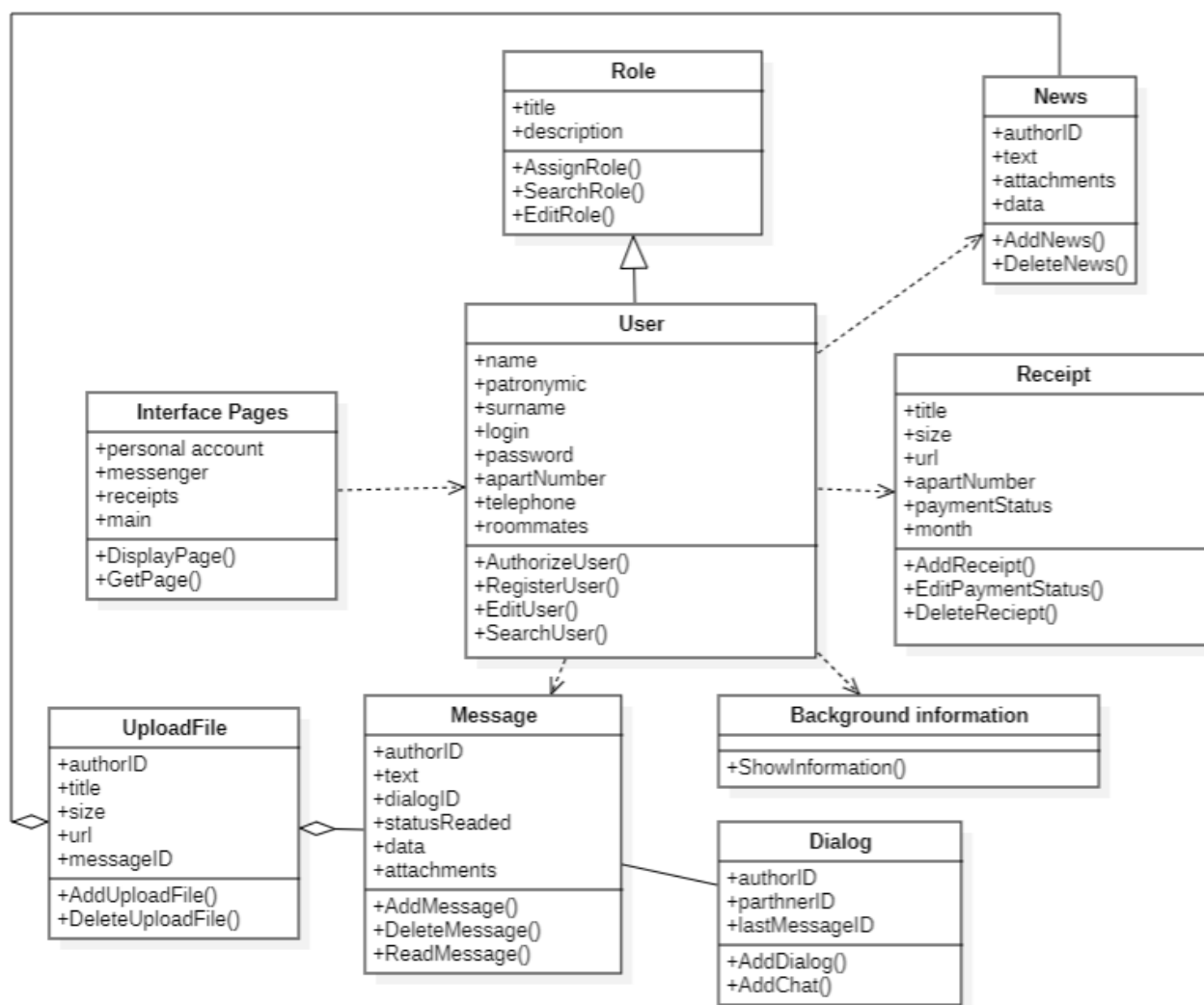


Рисунок 36 – Диаграмма классов (этап реализации)

3.3 Физическая модель данных

Физические модели данных служат для отображения моделей данных. Основными понятиями модели данных являются поле, логическая запись, логический файл. Слово «логический» введено, чтобы отличать понятия, относящиеся к логической модели данных от понятий, относящихся к физической модели данных.

Основными понятиями физической модели данных, используемыми для представления логической модели данных, являются поле, физическая запись, физический файл [31].

На рисунке 37 представлена физическая модель данных системы.

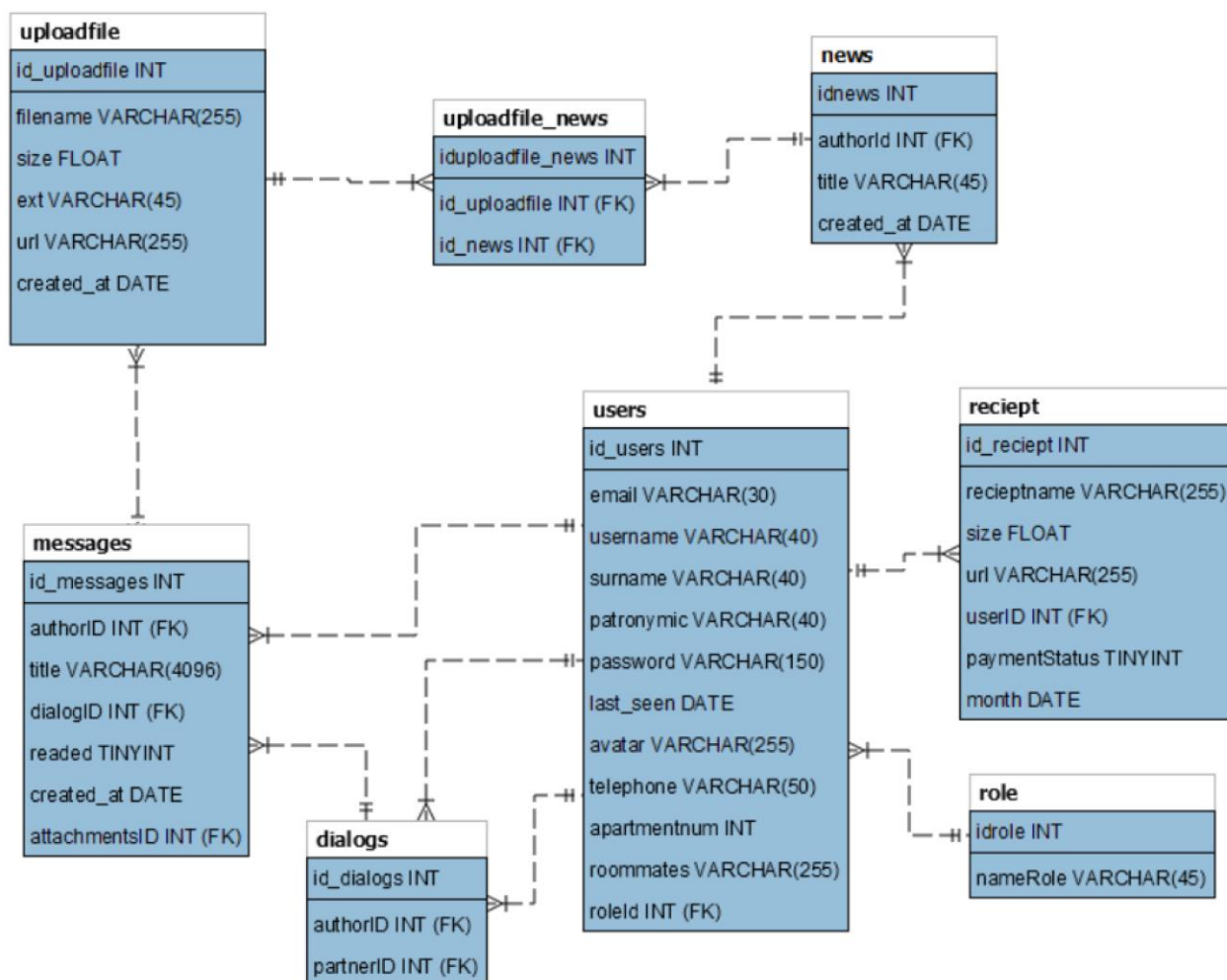


Рисунок 37 – Физическая модель данных

В таблицах 12 – 16 приведено описание сущностей БД. Первичные ключи выделены жирным шрифтом, а внешние – курсивом.

Таблица 12 – Сущность «Users»

Имя поля	Имя атрибута	Тип	Размер (байт)
ID	Идентификатор пользователя	Integer	4
email	Логин пользователя	Varchar(30)	60
username	Имя пользователя	Varchar(40)	80
surname	Фамилия пользователя	Varchar(40)	80
patronymic	Отчество пользователя	Varchar(40)	80
password	Пароль пользователя	Varchar(150)	300
last_seen	Последнее посещение пользователя	Date	3
avatar	Фотография пользователя	Varchar(255)	510
telephone	Номер телефона	Varchar(50)	100
apartmentnum	Номер квартиры	Integer	4
roommates	Сожители	Varchar(255)	510
roleId	Роль	Integet	4
ИТОГО			1735

Таблица 13 – Сущность «Reciept»

Имя поля	Имя атрибута	Тип	Размер (байт)
ID	Идентификатор типа квитанции	Integer	4
receiptname	Имя, используемое в качестве названия в системе	Varchar(45)	90
size	Объем памяти, который занимает квитанция	Varchar(45)	90
url	Ссылка на файл квитанции	Varchar(255)	510
<i>userID</i>	<i>Идентификатор собственника квартиры</i>	<i>Integer</i>	<i>4</i>
paymentStatus	Статус оплаты квитанции	Boolean	4
month	Месяц счета квитанции	Date	3
ИТОГО			713

Таблица 14 – Сущность «Dialogs»

Имя поля	Имя атрибута	Тип	Размер (байт)
ID	Идентификатор диалога	Integer	4
<i>authorid</i>	<i>Идентификатор автора диалога</i>	<i>Integer</i>	<i>4</i>
<i>partnerid</i>	<i>Идентификатор партнера диалога</i>	<i>Integer</i>	<i>4</i>
ИТОГО			12

Таблица 15 – Сущность «Role»

Имя поля	Имя атрибута	Тип	Размер (байт)
ID	Идентификатор роли	Integer	4
<i>namerole</i>	<i>Имя, используемое в качестве названия роли в системе</i>	<i>Varchar (45)</i>	<i>90</i>
ИТОГО			94

Таблица 16 – Сущность «Message»

Имя поля	Имя атрибута	Тип	Размер (байт)
ID	Идентификатор сообщения	Integer	4
<i>authorID</i>	<i>Идентификатор автора сообщения</i>	<i>Integer</i>	<i>4</i>
<i>title</i>	<i>Текст сообщения</i>	<i>Varchar (4096)</i>	<i>8192</i>
<i>dialogID</i>	<i>Идентификатор диалога</i>	<i>Integer</i>	<i>4</i>
<i>readed</i>	<i>Статус прочтения сообщения</i>	<i>Boolean</i>	<i>4</i>
<i>attachmentsId</i>	<i>Идентификатор файла</i>	<i>Integer</i>	<i>4</i>
<i>created_at</i>	<i>Дата отправки</i>	<i>Date</i>	<i>3</i>
ИТОГО			8219

Таблица 17 – Сущность «UploadFile»

Имя поля	Имя атрибута	Тип	Размер (байт)
1	2	3	4
ID	Идентификатор медиафайла	Integer	4
<i>filename</i>	<i>Имя, используемое в качестве названия в системе</i>	<i>Varchar(255)</i>	<i>510</i>

Продолжение таблицы 17

1	2	3	4
size	Объем памяти, который занимает медиафайл	Varchar(45)	90
ext	Расширение файла	Varchar(45)	90
url	Ссылка на медиафайл	Varchar(255)	510
created_at	Дата отправки	Date	3
ИТОГО			1207

Таблица 18 – Сущность «News»

Имя поля	Имя атрибута	Тип	Размер (байт)
ID	Идентификатор объявления	Integer	4
<i>authorID</i>	<i>Идентификатор автора объявления</i>	<i>Integer</i>	<i>4</i>
title	Текст объявления	Varchar(4096)	8192
created_at	Дата отправки	Date	3
ИТОГО			8203

Таблица 19 – Сущность «Uploadfile_News»

Имя поля	Имя атрибута	Тип	Размер (байт)
ID	Идентификатор связи	Integer	4
<i>ID_uploadfile</i>	<i>Идентификатор прикрепленного файла</i>	<i>Integer</i>	<i>4</i>
id_news	<i>Идентификатор объявления</i>	<i>Integer</i>	<i>4</i>
ИТОГО			12

3.4 Выбор и обоснование комплекса технических средств

3.4.1 Расчет объема занимаемой памяти

Расчет объема внешней памяти

Для расчета необходимого объема свободной внешней памяти, необходимой для функционирования системы, воспользуемся следующей формулой:

$$V_{\text{ЖД}} = V_{\text{ОС}} + V_{\text{ПР}} + V_{\text{СПО}} + V_{\text{БД}} + V_{\text{справки}},$$

где V_{OC} – объем памяти, занимаемый операционной системой (операционная система Windows 10, $V_{OC} = 5.5$ Гб);

$V_{ПР}$ – объем памяти, занимаемый непосредственно файлами приложения ($V_{ПР} = 571$ Кб);

$V_{СПО}$ – объем памяти, занимаемый сопутствующим программным обеспечением ($V_{СПО} = 134$ Мб);

$V_{БД}$ – объем памяти, занимаемый базой данных (всеми таблицами) при ее максимальном заполнении. (дадим оценку сверху $V_{БД}$ в 50 Мб);

$V_{справки}$ – объем памяти, необходимый для хранения файла справки, необходимого для работы программы (дадим ему оценку сверху в 1 Мб).

Таким образом, суммарный объем внешней памяти составит:

$$V_{ЖД} = 5.5 \text{ Гб} + 571 \text{ Кб} + 134 \text{ Мб} + 50 \text{ Мб} + 1 \text{ Мб} \sim 5.7 \text{ Гб}.$$

Расчет объема ОЗУ

Для расчета необходимого объема ОЗУ воспользуемся следующей формулой:

$$V_{ОЗУ} = V_{OC} + V_{ПР} + V_{БД} + V_{браузера},$$

где V_{OC} – ОЗУ, занимаемое операционной системой (1 Гб);

$V_{ПР}$ – ОЗУ, которое займет само приложение (не превысит 800 Мб);

$V_{БД}$ – объем данных из базы, который может быть одновременно загружен в оперативную память (дадим ему оценку сверху в 10 Мб);

$V_{браузера}$ – ОЗУ, занимаемое браузером (оценим его сверху значением в 100 Мб).

Суммарные объемы ОЗУ составит:

$$V_{ОЗУ} = 1 \text{ Гб} + 800 \text{ Мб} + 10 \text{ Мб} + 100 \text{ Мб} \sim 1.91 \text{ Гб}.$$

Таким образом, 1.91 Гб оперативной памяти можно считать минимально необходимым для функционирования системы.

3.4.2 Минимальные требования, предъявляемые к системе

Для корректного функционирования системы необходимо:

- тип ЭВМ: IBM PC совместимый;
- объем ОЗУ – не менее 1.91 Гб;
- объем свободного дискового пространства – не менее 5.7 Гб;
- клавиатура или иное устройство ввода;
- мышь или иное манипулирующее устройство;
- дисплей с разрешением не менее 1024×768 пикселей;
- операционная система – Windows 7 и выше;
- браузер – Google Chrome.

ЗАКЛЮЧЕНИЕ

В процессе выполнения выпускной квалификационной работы было разработано веб-приложение для управления многоквартирным домом, позволяющее работать с системой в двух разных ролях. Управляющая компания может создавать чаты с 3-мя и более участниками, контролировать оплату квитанций на оплату коммунальных услуг, создавать объявления. «Собственник» может создавать диалоги с 2-мя участниками, получать квитанции на оплату коммунальных услуг, просматривать объявления.

В первом разделе приведены основные понятия предметной области, описаны характеристики систем-аналогов, приведен их сравнительный анализ. На основе проведенного анализа выполнена объектная декомпозиция, отраженная в диаграмме объектов, и сформулирована постановка задачи.

Во втором разделе приведена структурная схема системы, определены выполняемые функции. Разработаны прототипы интерфейса пользователя, информационно-логический проект системы. С помощью языка UML построены канонические диаграммы. Кроме того, описан комплекс программных средств, использованных при разработке.

Третий раздел посвящен описанию пользовательского меню и тестовых примеров, конфигурации исполняемой программной системы. Выбран и обоснован комплекс технических средств, рассчитан объем необходимой памяти устройства для загрузки и работы с приложением, а также описаны минимальные требования, предъявляемые к системе.

Разработанная система позволит в реальном времени собственникам многоквартирных домов оперативно при помощи одного канала связи получать всю необходимую информацию о состоянии дома и придомовых территорий, коммуницировать с соседями, следить за оплатой квитанций ЖКХ, управляющим компании отправлять квитанции на оплату коммунальных платежей, создавать чаты и коммуницировать с собственниками жилья.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 Управляющая компания [Электронный ресурс]. URL: https://ru.wikipedia.org/wiki/Управляющая_компания (дата обращения: 15.11.2023).
- 2 Собственник [Электронный ресурс]. URL: <https://kartaslov.ru/значение-слова/собственник> (дата обращения: 15.11.2023).
- 3 Виды управляющих компаний. [Электронный ресурс]. URL: <https://j.etagi.com/ps/obyazannosti-uk/> (дата обращения: 15.11.2023).
- 4 Жилищный кодекс Российской Федерации. [Электронный ресурс]. URL: https://www.consultant.ru/document/cons_doc_LAW_51057/ (дата обращения: 15.11.2023).
- 5 Коммуникация управляющей компании с жителями. [Электронный ресурс]. URL: <https://digitaldeveloper.ru/blog/tpost/3fk89bx5n1-kommunikatsiya-upravlyayuschei-kompanii> (дата обращения: 18.05.2023).
- 6 Локоло [Электронный ресурс]. URL: <https://locolo.tatarstan.ru/> (дата обращения: 01.02.2024)
- 7 Все соседи [Электронный ресурс]. URL: <https://proptech.digitaldeveloper.ru/solutions/vse-sosedi> (дата обращения: 01.02.2024)
- 8 Госуслуги дом [Электронный ресурс]. URL: https://www.gosuslugi.ru/mp_dom (дата обращения: 01.02.2024)
- 9 Объектно-ориентированного подхода к проектированию [Электронный ресурс]. URL: https://www.kaznu.kz/content/files/news/folder22809/lec_4.pdf (дата обращения: 01.02.2024).
- 10 Требования к эффективности и надежности проектных решений [Электронный ресурс]. URL: https://studopedia.net/1_41759_trebovaniya-k-effektivnosti-i-nadezhnosti-proektnih-resheniy.html (дата обращения: 01.02.2024).

11 Архитектура информационных систем [Электронный ресурс]. URL: https://kpfu.ru/staff_files/F_1284155032/Arkhitektura_informacionnykh_sistem.pdf (дата обращения: 15.02.2024)

12 Структурный подход к проектированию ИС [Электронный ресурс]. URL: http://citforum.ru/database/case/glava2_1.shtml (дата обращения: 15.02.2024).

13 Интерфейс [Электронный ресурс]. URL: <https://blog.skillfactory.ru/glossary/interface/#:~:text=%D0%98%D0%BD%D1%82%D0%B5%D1%80%D1%84%D0%B5>. (дата обращения: 15.02.2024).

14 Prototyping in UX Design – Benefits & Types [Электронный ресурс]. URL: <https://www.geeksforgeeks.org/prototyping-in-ux-design-benefits-types/> (дата обращения: 15.02.2024).

15 Статья по Figma для начинающих веб-дизайнеров [Электронный ресурс]. URL: <https://tilda.education/articles-figma> (дата обращения: 15.02.2024).

16 Зеленко, Л.С. Методические указания к лабораторному практикуму по дисциплине «Программная инженерия». [Текст] / Л.С. Зеленко – Самара: СГАУ, 2012. – 21 с.

17 Диаграмма вариантов использования [Электронный ресурс]. URL: <http://khpi-iip.mipk.kharkiv.edu/library/case/leon/gl4/gl4.html> (дата обращения: 15.02.2024)

18 Функциональная спецификация [Электронный ресурс]. URL: https://ru.wikipedia.org/wiki/Функциональная_спецификация (дата обращения: 02.03.2024).

19 Диаграмма состояний [Электронный ресурс]. URL: http://it-gost.ru/articles/view_articles/97 (дата обращения: 02.03.2024).

20 Диаграмма деятельности [Электронный ресурс]. URL: <https://dic.academic.ru/dic.nsf/ruwiki/1299093> (дата обращения: 01.12.2023).

- 21 Унифицированный язык моделирования UML. Базовые понятия диаграмм классов [Электронный ресурс]. URL: http://it.kgsu.ru/UML/uml_0076.html (дата обращения: 05.03.2024).
- 22 Логическая модель данных [Электронный ресурс]. URL: <http://analyst.by/diagrams/logicheskaya-model-predmetnoy-oblasti> (дата обращения 05.03.2024).
- 23 JavaScript [Электронный ресурс]. URL: <https://ru.wikipedia.org/wiki/JavaScript> (дата обращения: 15.03.2024).
- 24 React [Электронный ресурс] URL: <https://medium.com/nuances-of-programming/плюсы-и-минусы-react-виртуальная-dom-синтаксис-jsx-и-другие-аргументы-для-спора-85a111ba1494> (дата обращения: 15.03.2024).
- 25 Visual Studio Code [Электронный ресурс]. URL: <https://code.visualstudio.com/Docs/editor/whyvscode> (дата обращения 15.03.2024).
- 26 PostgreSQL [Электронный ресурс]. URL: <https://www.postgresql.org/about/> (дата обращения 15.03.2024).
- 27 Windows [Электронный ресурс]. URL: <https://ru.wikipedia.org/wiki/Windows> (дата обращения 15.03.2024).
- 28 Диаграммы реализации [Электронный ресурс]. URL: <http://www.maksakov-sa.ru/ModelUML/DiagrReal/index.html> (дата обращения 10.05.2024).
- 29 Диаграмма компонентов [Электронный ресурс]. URL: https://sites.google.com/site/anisimovkhv/learning/pris/lecture/tema15/tema15_2 (дата обращения 10.05.2024).
- 30 Диаграмма развертывания [Электронный ресурс]. URL: https://studbooks.net/2430185/informatika/diagramma_programmnogo_obespecheniya (дата обращения 10.05.2024).
- 31 Физические модели данных [Электронный ресурс]. URL: <https://studfile.net/preview/2556063/page:21/> (дата обращения 10.05.2024).

ПРИЛОЖЕНИЕ А

Руководство пользователя

А.1 Назначение системы

Система предназначена на управления многоквартирным домом в режиме реального времени. С помощью системы пользователю в роли «Собственник» доступны функции: получить квитанцию на услуги ЖКХ, перейти в личный кабинет, перейти в мессенджер, просмотреть ленту объявлений, отправить медиафайл, текстовое и голосовое сообщение. В роли «Управляющая компания» система позволяет пользователю загрузить квитанции на услуги ЖКХ, создать чат, создать новостное объявление, отправить медиафайл, текстовое и голосовое сообщение.

А.2 Условия работы системы

Для корректной работы системы необходимо наличие соответствующих программных и аппаратных средств.

1) Требования к техническому обеспечению:

- ЭВМ типа IBM PC совместимый;
- объем ОЗУ – не менее 1.91 Гб;
- объем свободного дискового пространства – не менее 5,7 Гб;
- клавиатура или иное устройство ввода;
- мышь или иное манипулирующее устройство;
- дисплей с разрешением не менее 1024×768 пикселей.

2) Требования к программному обеспечению:

- операционная система: Windows 7 и выше;
- браузер Google Chrome.

А.3 Работа с системой

А.3.1 Начало работы с системой

Для того, чтобы начать работу с системой, необходимо войти на сайт, в результате чего откроется главная страница неавторизованного пользователя (рисунок А.1).



Рисунок А.1 – Главная страница неавторизованного пользователя

А.3.2 Работа с системой в режиме пользователя «Собственник»

После успешной авторизации пользователь попадает на главную страницу авторизованного пользователя веб-приложения. С этой страницы пользователь может перейти к просмотру справочной информации, к страницам «Мессенджер», «Личный кабинет», «Квитанции», «Справочная информация», а также выйти из системы (рисунок А.2).

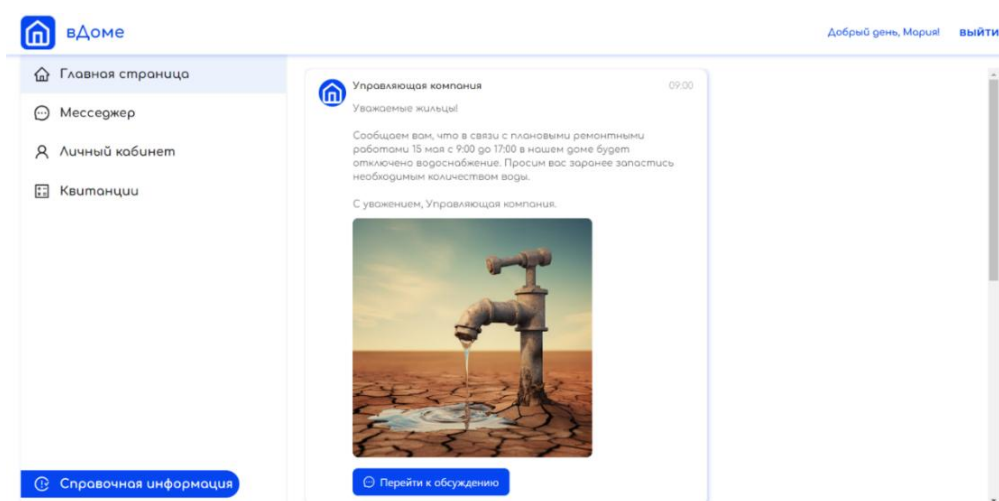


Рисунок А.2 – Главная страница авторизованного пользователя

При нажатии на кнопку «Мессенджер» открывается страница с мессенджером веб-приложения. В верхней части веб-страницы находится шапка веб-приложения с доступными кнопками «Выйти» и «Логотип», по нажатию на который пользователь перейдет на главную страницу. С левой стороны веб-страницы расположен список доступных пользователю чатов, кнопки «Создать чат» и «Поиск диалога». С правой стороны веб-

страницы находится текущий диалог с текстовым полем для ввода нового сообщения снизу, названием чата сверху. С этой страницы пользователь может перейти главной странице, а также выйти из системы (рисунок А.3).

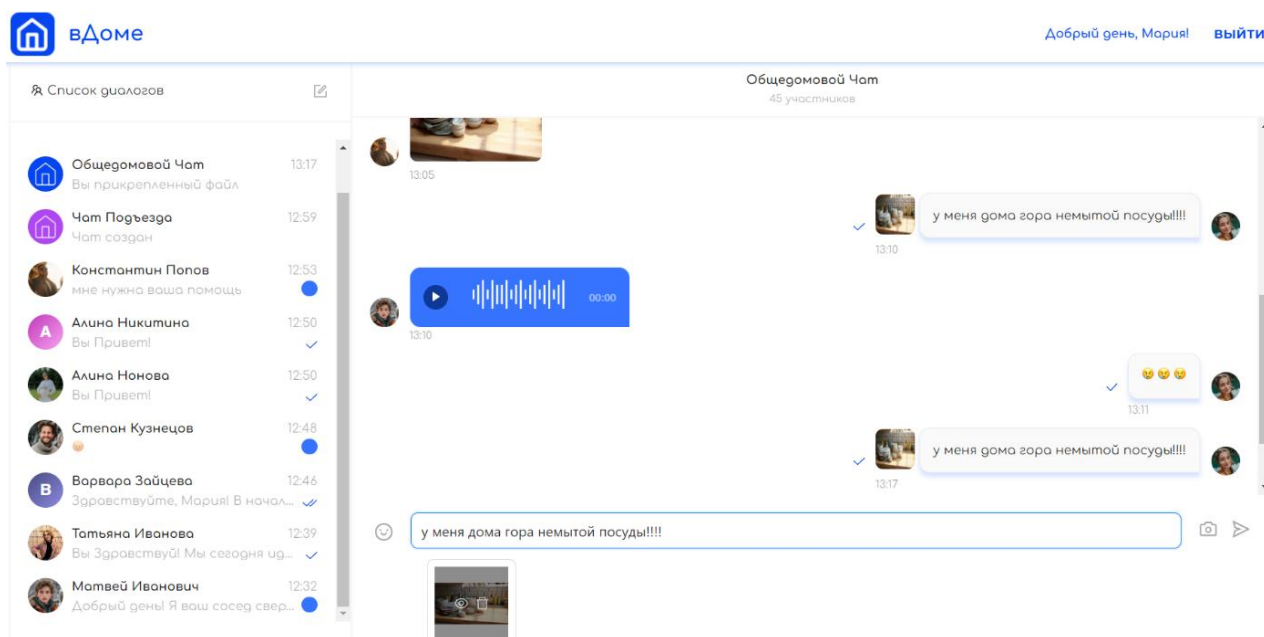


Рисунок А.3 – Страница «Мессенджер»

При нажатии на кнопку «Личный кабинет» открывается страница личного кабинета пользователя. На данной странице пользователь может изменить фотографию и личные данные: номер телефона и совместно проживающих граждан. С этой страницы пользователь может перейти на страницы «Главная», «Мессенджер», «Квитанции», «Справочная информация» и выйти из системы (рисунок А.4).

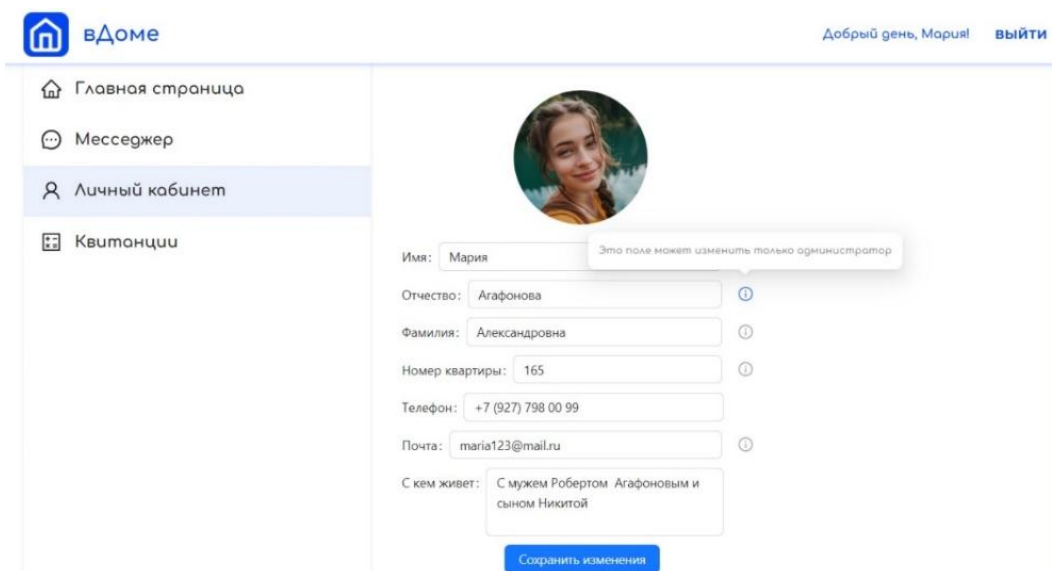


Рисунок А.4 – Страница «Личный кабинет»

При нажатии на кнопку «Квитанции» открывается страница со списком доступных квитанций. С этой страницы пользователь может перейти на страницы «Главная», «Мессенджер», «Личный кабинет», «Справочная информация» и выйти из системы (рисунок А.5).

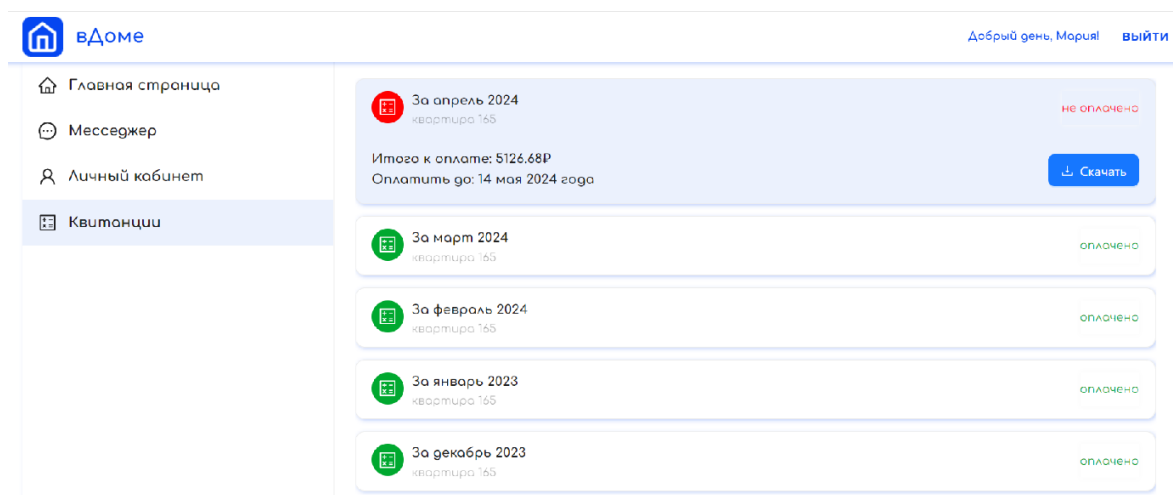


Рисунок А.5 – Страница «Квитанции»

А.3.3 Работа с системой в режиме пользователя «Управляющая компания»

После успешной авторизации пользователь в роли «Управляющая компания», также как и «Собственник», попадает на главную страницу авторизованного пользователя веб-приложения. Однако такому пользователю доступен больший функционал. На главной странице авторизованного пользователя «Управляющая компания» может также добавлять новые объявления (рисунок А.6).

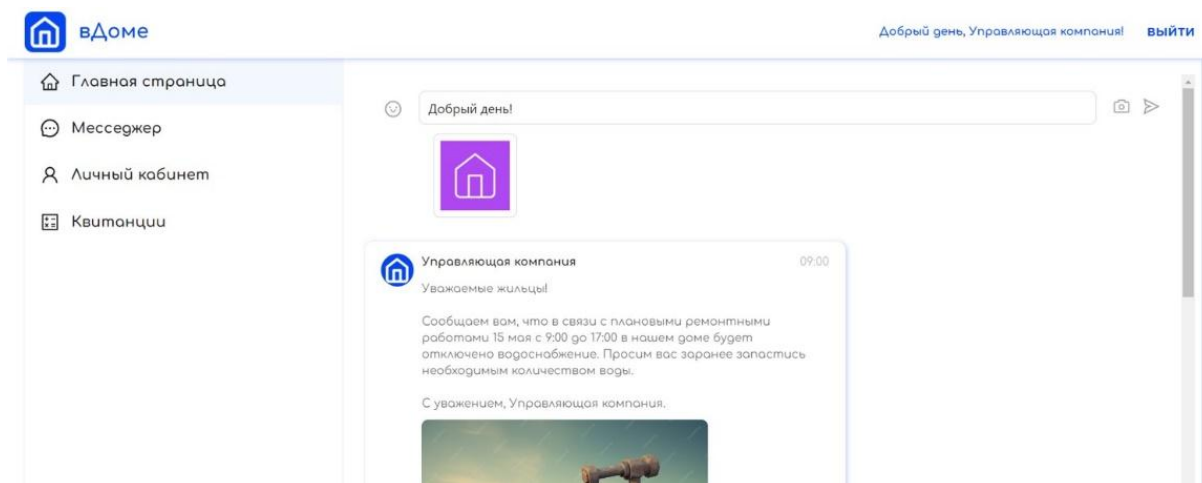


Рисунок А.6 – Главная страница авторизованного пользователя «Управляющая компания»

Пользователь в роли «Управляющая компания» на странице «Мессенджер» может создавать чаты с тремя и более пользователями. Для этого необходимо выбрать собеседников, ввести текстовое сообщение и название чата (рисунок А.7).

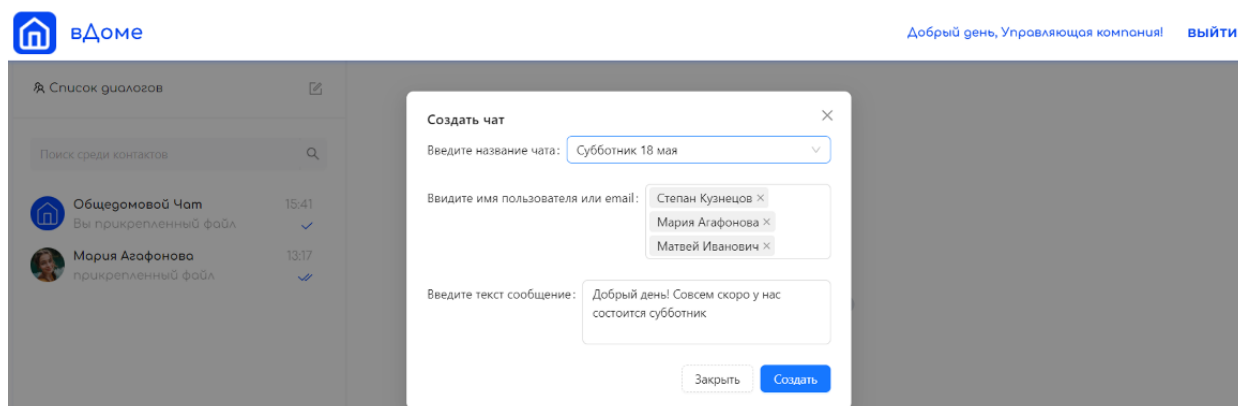


Рисунок А.7 – Страница «Мессенджер» авторизованного пользователя
«Управляющая компания»

Также на странице «Квитанции» пользователь может загружать новые квитанции и изменять статус оплаты (рисунок А.8).

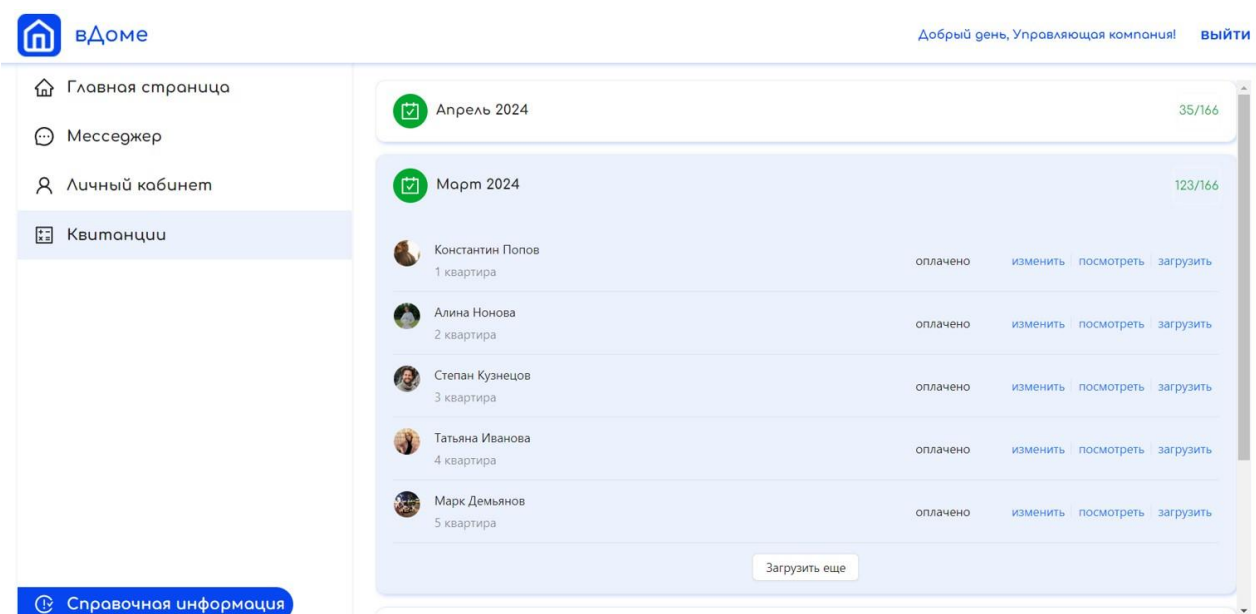


Рисунок А.8 – Страница «Квитанции» авторизованного пользователя
«Управляющая компания»

А.3.4 Основные ошибки

Если при заполнении формы регистрации ввести некорректные данные, не соответствующие ожидаемым требованиям системы, пользователь увидит сообщение об ошибке (рисунок А.9).

Регистрация

Введите данные для регистрации

Имя*

11

Имя должно быть указано в корректном формате

Отчество*

Дмитриевна

Фамилия*

Топыркина

Email*

ttt1971@yandex.ru

Пароль*

.....

Пароль слишком короткий

Повторите пароль*

.....

Повторите пароль

РЕГИСТРАЦИЯ

[У вас есть аккаунт? Авторизация](#)

Рисунок А.9 – Ошибка формы регистрации пользователя

При заполнении формы авторизации и введении некорректных данных, не соответствующих требованиям системы, также появится сообщение об ошибке (рисунок А.10).

Авторизация

Введите данные для авторизации

Email*

maria123@mail.ru

Пароль*

.....

Пароль слишком короткий

Рисунок А.10 – Ошибка формы авторизации пользователя

Если после заполнения формы авторизации логин и пароль окажутся неверными, появится окно с сообщением «Ошибка при авторизации. Неверный логин или пароль» (рисунок А.11).

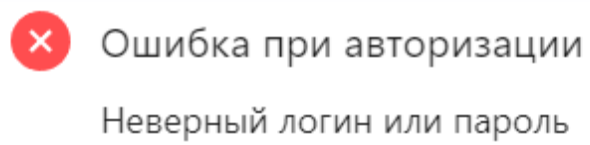


Рисунок А.11 – Ошибка авторизации пользователя

ПРИЛОЖЕНИЕ Б

Листинг модулей программы

Серверная часть:

```
const express = require('express')
const dotenv = require('dotenv')
const {createServer} = require('http')
const path = require('path')

dotenv.config()

const userRouter = require('./routes/user.routes')
const dialogRouter = require('./routes/dialog.routes')

const uploadRouter = require('./routes/upload.routes')
const messageRouter = require('./routes/message.routes')
const corsMiddleware = require('./middleware/cors.middleware')
const authMiddleware = require('./middleware/checkAuth.middleware')
//const updateLastSeen = require('./middleware/updateLastSeen.middleware')

const createSocket = require('./core/io.socket')

const app = express()
const http = createServer(app)
const io = createSocket(http);

app.use(corsMiddleware)
app.use(authMiddleware)
//app.use(updateLastSeen)
app.use(express.json())
app.use('/images', express.static(path.join(__dirname, 'images')))

const start = async () => {
  try {
    app.use('/api/auth', userRouter(io))
    app.use('/api/chat', dialogRouter(io))
    app.use('/api/chat', messageRouter(io))
    app.use('/api', uploadRouter(io))

    /*io.on('connection', function(socket) {
      console.log('CONNECTED!');
    });*/

    http.listen(process.env.PORT, () => {
      console.log(`Server: http://localhost:${process.env.PORT}`)
    })

  } catch (e) {

  }
}

start()
const db = require('../db')
```

```

class DialogController {
  constructor(io) {
    this.io = io;
  }
  io;

  /*
  вот тут я получаю только данные о диалоге, это может быть нужно

  ниже я получаю данные о диалоге, авторе, партнеру

  async index(req, res) {
    try {
      const authorId = req.params.id;
      const dialogs = await db.query('SELECT * FROM dialogs WHERE authorId = $1', [authorId]);
      if (dialogs.length === 0) {
        return res.status(404).json({
          message: 'Dialogs not found',
        });
      }
      res.json(dialogs);
    } catch (error) {
      console.error(error)
      res.status(500).json({ message: 'Internal Server Error', error: error.message })
    }
  };
  */

  async index(req, res) {
    try {
      const emailId = req.user.data
      const user = await db.query(`SELECT * FROM users WHERE email = '${emailId}' `);

      const dialogs = await db.query('SELECT * FROM dialogs WHERE authorId = $1 OR partnerId = $1',
[user.rows[0].id]);

      if (dialogs.length === 0) {
        return res.status(404).json({
          message: 'Dialogs not found',
        });
      }

      // Для каждого диалога получаем полные данные о пользователях
      const dialogsWithUsers = await Promise.all(dialogs.rows.map(async (dialog) => {
        const author = await db.query('SELECT * FROM users WHERE id = $1', [dialog.authorid]);
        const partner = await db.query('SELECT * FROM users WHERE id = $1', [dialog.partnerid]);
        const lastMessage = await db.query('SELECT * FROM messages WHERE id = $1', [dialog.lastmessageid]);
        return {
          id: dialog.id,
          author: author.rows[0],
          partner: partner.rows[0],
          lastMessage: lastMessage.rows[0],
        };
      }));

      return res.json(dialogsWithUsers);
    } catch (error) {

```

```

        console.error(error);
        res.status(500).json({ message: 'Internal Server Error', error: error.message });
    }
}

create = async (req, res) =>{
    try {

        const { partnerid, title } = req.body;
        const emailID = req.user.data;
        const user = await db.query(`SELECT * FROM users WHERE email = '${emailID}'`);
        // Проверяем, существует ли уже диалог между автором и партнером
        const existingDialog = await db.query(
            `SELECT EXISTS (SELECT 1 FROM dialogs WHERE (authorId = '${user.rows[0].id}' AND partnerId =
            '${partnerid}') OR (authorId = '${partnerid}' AND partnerId = '${user.rows[0].id}')) AS it_does_exist;`
        );

        if (existingDialog.rows[0].it_does_exist) {
            return res.status(400).json({ message: 'Dialog already exists' });
        }

        // Вставляем новый диалог в таблицу dialogs
        const newDialog = await db.query(
            `INSERT INTO dialogs(authorId, partnerId) VALUES ('${user.rows[0].id}', '${partnerid}') RETURNING *`
        );

        // Вставляем новое сообщение в таблицу messages для установки lastMessage
        const newMessage = await db.query(
            `INSERT INTO messages (title, dialogid, authorid, readed, created_at) VALUES ($1, $2, $3, $4, $5)
            RETURNING *`,
            [title, newDialog.rows[0].id, user.rows[0].id, false, new Date()]
        );

        // Обновляем lastMessage в созданном диалоге
        await db.query(
            `UPDATE dialogs SET lastMessageId = $1 WHERE id = $2`,
            [newMessage.rows[0].id, newDialog.rows[0].id]
        );

        this.io.emit('SERVER:DIALOG_CREATED', {
            partner: partnerid,
            dialog: {
                id: newDialog.rows[0].id,
                authorid: newDialog.rows[0].authorid,
                partnerid: newDialog.rows[0].partnerid,
                lastMessage: {
                    id: newMessage.rows[0].id,
                    authorid: user.rows[0].id,
                    text: newMessage.rows[0].title,
                    readed: false,
                    created_at: newMessage.rows[0].created_at
                }
            }
        });

        res.json({

```

```

        partner: partnerid,
        dialog: {
          id: newDialog.rows[0].id,
          authorid: newDialog.rows[0].authorid,
          partnerid: newDialog.rows[0].partnerid,
          lastmessageid: {
            id: newMessage.rows[0].id,
            authorid: user.rows[0].id,
            readed: false,
            created_at: newMessage.rows[0].created_at
          }
        }
      });

    } catch (error) {
      console.error('Error creating dialog:', error);
      res.status(500).json({
        message: 'Internal server error',
      });
    }
  };

deleteDialog = async (req, res) => {
  try {
    const id = req.params.id;

    // Удаляем диалог по id из таблицы dialogs
    const deletedDialog = await db.query('DELETE FROM dialogs WHERE id = $1 RETURNING *', [id]);

    if (deletedDialog) {
      res.json({
        message: 'Dialog deleted'
      });
    } else {
      res.status(404).json({
        message: 'Dialog not found'
      });
    }
  } catch (error) {
    console.error('Error deleting dialog:', error);
    res.status(500).json({
      message: 'Internal server error'
    });
  }
};

}

module.exports = DialogController;

const db = require('../db')
const bcrypt = require('bcryptjs')
const createJWTToken = require('../utils/createJWTToken')

class UserController {
  io;

```

```

constructor(io) {
  this.io = io;
}

createUser = async (req, res) => {
  try {
    const { email, username, surname, patronymic, password } = req.body
    const candidate = await db.query(`SELECT EXISTS (SELECT 1 FROM users WHERE email = '${email}') AS
it_does_exist;`)
    if (candidate.rows[0].it_does_exist) {
      return res.status(400).json({ message: `User with email ${email} already exist` })
    }
    const hashPassword = await bcrypt.hash(password, 8)
    const newPerson = await db.query(`INSERT INTO users(email, username, surname, patronymic, password,
last_seen) values ($1, $2, $3, $4, $5, $6) RETURNING *`,
[email, username, surname, patronymic, hashPassword, new Date()])
    //res.json(newPerson.rows[0])
    const status = "success"
    return res.json({ status });
  } catch (error) {
    console.error(error)
    res.status(500).json({ message: 'Internal Server Error', error: error.message })
  }
}

getUsers = async (req,res) => {
  try {
    const users = await db.query(`SELECT * FROM users`)
    res.json(users.rows)
  } catch (error) {
    console.error(error)
    res.status(500).json({ message: 'Internal Server Error', error: error.message })
  }
}

getUser = async (req,res) => {
  try {
    const id = req.params.id
    const users = await db.query(`SELECT * FROM users WHERE id = '${id}'`)
    res.json(users.rows[0])
  } catch (error) {
    console.error(error)
    res.status(500).json({ message: 'Internal Server Error', error: error.message })
  }
}

findUser = async(req,res) => {
  try {
    const query = req.query.query
    console.log(query);
    const users = await db.query(`SELECT * FROM users WHERE email LIKE $1 OR username LIKE $1',
['%${query}%`]);
    res.json(users.rows)
  } catch (error) {
    console.error(error)
    res.status(404).json({ status: 'error', message: 'Internal Server Error', error: error.message })
  }
}

```

```

getme = async(req,res) => {
  try {
    const email = req.user.data
    const users = await db.query(`SELECT * FROM users WHERE email = '${email}' `)
    res.json( users.rows[0])
  } catch (error) {
    console.error(error)
    res.status(500).json({ message: 'Internal Server Error', error: error.message })
  }
}

```

```

updateUser = async (req,res) => {
  try {
    const email = req.user.data // получаем email по токену
    const user = await db.query(`SELECT * FROM users WHERE email = '${email}' `)
    let { telephone, roommates, avatar } = req.body

    if (telephone == null) {
      telephone = user.rows[0].telephone
    }
    if (roommates == null) {
      roommates = user.rows[0].roommates
    }
    if (avatar == null) {
      avatar = user.rows[0].avatar
    }

    const users = await db.query(`UPDATE users set telephone = '${telephone}', roommates = '${roommates}',
    avatar = '${avatar}' where id = '${user.rows[0].id}' RETURNING * `)
    res.json(users.rows[0])
  } catch (error) {
    console.error(error)
    res.status(500).json({ message: 'Internal Server Error', error: error.message })
  }
}

```

```

deleteUser = async (req,res) => {
  try {
    const id = req.params.id
    const user = await db.query(`DELETE FROM users where id = '${id}' `)
    res.json(user.rows[0])
  } catch (error) {
    console.error(error)
    res.status(500).json({ message: 'Internal Server Error', error: error.message })
  }
}

```

```

loginUser = async (req, res) => {
  try {
    const { email, password } = req.body;
    const user = await db.query(`SELECT * FROM users WHERE email = $1`, [email]);
    if (user.rows.length === 0) {
      return res.json({ status: "error" });
    }
  }
}

```

```

const isPassValid = bcrypt.compareSync(password, user.rows[0].password);

```

```

    if (!isPassValid) {
      return res.json({ status: "error" });
    }

    db.query(`UPDATE users SET last_seen = $1 WHERE email = $2`, [new Date(), email]);
    const status = "success"
    const token = createJWTToken(email);

    return res.json({
      token, status, user: user.rows[0]
    });
  } catch (error) {
    console.error(error);
    res.status(500).json({ message: "Internal Server Error", error: error.message });
  }
}
}
}

```

Клиентская часть:

```

import React, {useEffect} from 'react';
import withRouter from '../utils/helpers/withRouter';
import { connect } from 'react-redux';

```

```

import dialogsActions from '../reducers/actions/dialogs'
import Sidebar from '../containers/Sidebar';
import Status from '../containers/Status';
import Messages from '../containers/Messages';
import Chatinput from '../containers/ChatInput';
import './HomeChat.scss';

```

```

const HomeChat = (props) => {
  const {setCurrentDialogId, user} = props;

```

```

  useEffect(() => {
    const { location: { pathname } } = props;
    const dialogId = pathname.split('/').pop();
    if (dialogId !== 'chat'){
      setCurrentDialogId(dialogId);
    } else {
      setCurrentDialogId("");
    }
  }

```

```

  }, [props.location.pathname]);

```

```

  return (
    <section className="home">

      <div className="chat">
        <Sidebar />
        { user && (<div className="chat__dialog">
          <Status online/>
          <Messages />
          <div className="chat__dialog-input">
            <Chatinput />
          </div>
        </div>)}
      </div>
    </div>
  )

```


</section>

```
);  
}
```

```
export default withRouter(connect(  
  ({ user }) => ({ user: user.data}),  
  dialogsActions  
)(HomeChat)  
);
```

```
import React, { useEffect, useState, useRef } from "react";  
import { connect } from 'react-redux';  
import { Empty } from 'antd';  
import PropTypes from "prop-types";
```

```
import messagesActions from '../reducers/actions/messages';  
import BaseMessages from '../components/chat/Messages/Messages';  
import socket from '../core/socket';
```

```
const Dialogs = ({  
  currentDialogId,  
  fetchMessages,  
  addMessage,  
  items,  
  isLoading,  
  removeMessageById,  
  user,  
  attachments  
}) => {  
  if (!currentDialogId) {  
    return <Empty description="Начните диалог" />;  
  }  
}
```

```
const [previewImage, setPreviewImage] = useState(null);  
const [blockHeight, setBlockHeight] = useState(140);  
const [isTyping, setIsTyping] = useState(false);  
const typingTimeoutId = null;
```

```
const messagesRef = useRef(null);
```

```
const onNewMessage = data => {  
  addMessage(data);  
};
```

```
const toggleIsTyping = () => {  
  setIsTyping(true);  
  clearInterval(typingTimeoutId);  
  typingTimeoutId = setTimeout(() => {  
    setIsTyping(false);  
  }, 3000);  
};
```

```
useEffect(() => {  
  // socket.on('DIALOGS:TYPING', toggleIsTyping)  
}, []);
```

```

useEffect(() => {
  if (attachments.length) {
    setBlockHeigth(245);
  } else {
    setBlockHeigth(140);
  }
}, [attachments]);

useEffect(() => {
  if (currentDialogId) {
    fetchMessages(currentDialogId);
  }

  socket.on('SERVER:NEW_MESSAGE', onNewMessage);

  return () => socket.removeListener('SERVER:NEW_MESSAGE', onNewMessage);
}, [currentDialogId]);

useEffect(() => {
  messagesRef.current.scrollTo(0, 999999);
}, [items]);

return (
  <BaseMessages
    user={user}
    blockRef={messagesRef}
    items={items}
    isLoading={isLoading && !user}
    onRemoveMessage={removeMessageById}
    setPreviewImage={setPreviewImage}
    previewImage={previewImage}
    blockHeigth={blockHeigth}
    isTyping={isTyping}/>
);
};

export default connect(
  ({ dialogs, messages, user, attachments }) => ({
    currentDialogId: dialogs.currentDialogId,
    items: messages.items,
    isLoading: messages.isLoading,
    attachments: attachments.items,
    user: user.data
  }),
  messagesActions
)(Dialogs);
import React, { useState } from "react";
import { connect } from 'react-redux';
import userAPI from '../core/user';
import dialogsAPI from '../core/dialogs';

import Sidebar from '../components/chat/Sidebar/Sidebar';

const SidebarContainer = ({ user }) => {
  const [visible, setVisible] = useState(false);
  const [inputValue, setInputValue] = useState("");
  const [messageText, setMessageText] = useState("");
  const [users, setUsers] = useState([]);

```

```

const [isLoading, setIsLoading] = useState(false);
const [selectedUserId, setSelectedUserId] = useState(false);

const handleChangeInput = (value) => {
  setInputValue(value)
};

const onClose = () => {
  setVisible(false);
};

const onShow = () => {
  setVisible(true);
};

const onSearch = (value) => {
  setIsLoading(true);
  userAPI
    .findUser(value)
    .then(({data}) => {
      setUsers(data);
      setIsLoading(false);
    }).catch(() => {
      setIsLoading(false);
    });
};

const onAddDialog = () => {
  dialogsAPI
    .create(messageText, selectedUserId)
    .then(({data}) => {
      console.log(data);
      setIsLoading(false);
      onClose();
    }).catch(() => {
      setIsLoading(false);
    });
};

const onChangeTextArea = e => {
  setMessageText(e.target.value);
};

const onSelectUser = userId => {
  console.log(userId);
  setSelectedUserId(userId);
};

return (
  <Sidebar
    user={user}
    inputValue={inputValue}
    visible={visible}
    isLoading={isLoading}
    onClose={onClose}
    onShow={onShow}
    onChangeInput={handleChangeInput}
  >

```

```

      onSelectUser={onSelectUser}
      onSearch={onSearch}
      onModalOk={onAddDialog}
      onChangeTextArea={onChangeTextArea}
      messageText={messageText}
      selectedUserId={selectedUserId}
      users={users}
    />
  );
};

export default connect(({ user }) => ({ user: user.data }))(SidebarContainer);

import React, { useEffect, useRef, useState } from 'react';
import PropTypes from 'prop-types';
import ClassNames from 'classnames';
import format from 'date-fns/format';
import isToday from 'date-fns/isToday';
import { Popover, Button } from 'antd';
import { EllipsisOutlined, EyeOutlined } from '@ant-design/icons';
import reactStringReplace from 'react-string-replace';

import { convertCurrentTime, isAudio } from '../utils/helpers';

import waveSvg from "../assets/img/wave.svg";
import pauseSvg from "../assets/img/pause.svg";
import playSvg from "../assets/img/play.svg";

import './Message.scss';
import IconReaded from '../IconReaded/IconReaded';
import Avatar from '../Avatar/Avatar';

const getMessageTime = created_at => {

  const dateObject = new Date(created_at);

  return isToday(dateObject)
    ? format(dateObject, 'HH:mm')
    : format(dateObject, 'dd.MM.yyyy');
};

const MessageAudio = ({ audioSrc }) => {

  const audioElem = useRef(null);
  const [isPlaying, setIsPlaying] = useState(false);
  const [progress, setProgress] = useState(0);
  const [currentTime, setCurrentTime] = useState(0);

  const togglePlay = () => {
    if (!isPlaying) {
      audioElem.current.play();
    } else {
      audioElem.current.pause();
    }
  };

  useEffect(() => {
    audioElem.current.addEventListener(

```

```

    'playing',
    () => {
      setIsPlaying(true);
    },
    false,
  );
  audioElem.current.addEventListener(
    'ended',
    () => {
      setIsPlaying(false);
      setProgress(0);
      setCurrentTime(0);
    },
    false,
  );
  audioElem.current.addEventListener(
    'pause',
    () => {
      setIsPlaying(false);
    },
    false,
  );
  audioElem.current.addEventListener('timeupdate', () => {
    const duration = (audioElem.current && audioElem.current.duration) || 0;
    setCurrentTime(audioElem.current.currentTime);
    setProgress((audioElem.current.currentTime / duration) * 100);
  });
}, []);

return (
  <div className="message__audio">
    <audio ref={audioElem} src={audioSrc} preload="auto" />

    <div className="message__audio-progress" style={{ width: progress.toString() + '%' }} />
    <div className="message__audio-info">
      <div className="message__audio-btn">
        <button type="button" onClick={togglePlay}>
          {isPlaying ? (
            <img src={pauseSvg} alt="Pause svg" />
          ) : (
            <img src={playSvg} alt="Play svg" />
          )}
        </button>
      </div>
      <div className="message__audio-wave">
        <img src={waveSvg} alt="Wave svg" />
      </div>
      <span className="message__audio-duration">{convertCurrentTime(currentTime)}</span>
    </div>
  </div>
);
};

const Message = ({
  user,
  text,
  created_at,
  isMe,
  readed,

```

```

attachments,
isTyping,
onRemoveMessage,
setPreviewImage
}) => {

const renderAttachment = item => {
  if(item.ext !== 'webm') {
    return (
      <div onClick={() => setPreviewImage(item.url)} key={item.id} className="message__attachments-item">
        <div className='message__attachments-item-overlay' >
          <EyeOutlined style={{ color : 'white', fontSize: 18 }} />
        </div>
        <img src={item.url} alt={item.filename} />
      </div>
    )
  } else {
    return <MessageAudio key={item.id} audioSrc={item.url}/>
  }
};

```

```

return (
  <div className={ClassNames('message', {
    'message--is-me': isMe,
    'message--is-typing': isTyping,
    'message--is-audio': isAudio(attachments),
    'message--image':
      !isAudio(attachments) && attachments && attachments.length === 1 && !text
  })}>
    <div className='message__content'>
      <Popover
        content={
          <div>
            <Button onClick={onRemoveMessage}>Удалить сообщение</Button>
          </div>
        }
        trigger="click">
        <div className='message__icon-actions'>
          <Button type="ghost" shape="circle" icon={<EllipsisOutlined style={{ fontSize: '22px' }} />} />
        </div>
      </Popover>

      <IconReaded isMe={isMe} isReaded={readed} />
      <div className='message__avatar'>
        <Avatar user={user} />
      </div>
      <div className="message__info">
        { (text || isTyping) && (
          <div className="message__bubble">
            {text && (<p className="message__text">
              {reactStringReplace(text, /:(.+?):/g, (match,i) => (
                <em-emoji id={match} set="apple"></em-emoji>
              ))}</p>)}
            {isTyping && (
              <div className="message__typing">
                <span />

```

```

        <span />
        <span />
    </div>
  )}
  { false && ( <MessageAudio audioSrc={null}/> )}
</div>
)}

{ attachments && (
  <div className="message__attachments">
    {attachments.map((item, index) => renderAttachment(item))}
  </div>
  )}
</div>
{created_at &&
  <span className="message__date">
    {getMessageTime(created_at)}
  </span> }
</div>
</div>
});

Message.defaultProps = {
  user: {},
  avatar: "", // Значение по умолчанию для "avatar"
  text: "", // Значение по умолчанию для "text"
  date: "", // Значение по умолчанию для "date"
  isMe: false, // Значение по умолчанию для "isMe"
  isReaded: false, // Значение по умолчанию для "isReaded"
  attachments: [], // Значение по умолчанию для "attachments"
  isTyping: false, // Значение по умолчанию для "isTyping"
  audio: ""
};

Message.propTypes = {
  avatar: PropTypes.string,
  text: PropTypes.string,
  date: PropTypes.string,
  user: PropTypes.object,
  isMe: PropTypes.bool,
  isReaded: PropTypes.bool,
  attachments: PropTypes.array,
  isTyping: PropTypes.bool,
  audio: PropTypes.string,
};

export default Message;

```